

Лабораторная работа №2

Модель SIR. Модель Лотки–Вольтерры

Люпп Софья Романовна

Содержание

1. Цель работы

Изучить модель SIR, модель Лотки–Вольтерры, узнать ее практическое применение, изучить трехпараметрическую модель и ее практическую ценность.

2. Задание

- 1) Создать и запустить программный код с моделью SIR;
- 2) Создать и запустить программный код модели Лотки–Вольтерры;
- 3) Выполнить практическое задание в конце лабораторной работы.

3. Теоретическое введение

Модель SIR делит всю популяцию на три взаимосвязанные группы, что отражено в её названии: — S — Susceptible: люди, которые не болели, не имеют иммунитета и могут заразиться. — I — Infectious: люди, которые в данный момент больны и могут передавать инфекцию. — R — Recovered: люди, которые переболели и приобрели иммунитет (или умерли).

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

Практическое применение модели SIR: обучения основам эпидемиологии, быстрой оценки масштаба угрозы по R_0 , прогнозирования нагрузки на систему здравоохранения (пиковое значение I , оценки эффекта мер: карантин, изоляция, маски снижают параметр β).

В классической двухпараметрической модели (β , γ) параметр β (коэффициент заражения) является составным. Он скрывает в себе два процесса: контакт между людьми (поведенческий, управляемый фактор) и передачу инфекции при контакте (биологический фактор). Трёхпараметрическая модель делает это разделение явным: где c — среднее число контактов (достаточно тесных для передачи инфекции) одного человека в единицу времени, β — вероятность передачи инфекции при одном контакте между заразным и восприимчивым (безразмерная величина, от 0 до 1) и γ — скорость выздоровления (доля инфицированных, выздоравливающих в единицу времени). Как и раньше, $1/\gamma$ — средняя продолжительность заразного периода.

Практическая ценность трёхпараметрической формы заключается в планировании мер: позволяет оценить эффективность разных стратегий, а также она более наглядна для неспециалистов.

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв.

Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

4. Выполнение лабораторной работы

Начиная с изучения теории по модели SIR. Далее необходимо создать новый файл `sir_ode` с кодом для анализа результатов общей численности популяции, пикового числа зараженных, времени достижения пика, итогового числа и доли переболевших (рис. 1) (рис. 2) (рис. 3).

```
srliipp@vbox:~$ cd project/scripts
srliipp@vbox:~/project/scripts$ touch sir_ode.jl
srliipp@vbox:~/project/scripts$ gedit sir_ode.jl
srliipp@vbox:~/project/scripts$ julia sir_ode.jl
```

Рисунок 1: *siranalysis*

```
Open [v] [x]
138 hline!(plt6, [1.0], color=:red, linestyle=:dash, label="Порог эпидемии ( $R_0=1$ )", linewidth=1.5)
139 # Отметка момента, когда  $R_0$  становится < 1
140 cross_idx = findfirst(x -> x < 1, df_ode.Re)
141 if !isnothing(cross_idx) && cross_idx > 1
142 cross_time = df_ode.t[cross_idx]
143 vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=false, linewidth=1)
144 annotate!(plt6, cross_time, 1.2,
145 text(" $R_0 < 1$  с  $t = \text{round}(\text{cross\_time}, \text{digits}=1)$  дня", 8, :left))
146 end
147 # Компактный график всех кривых в одной панели
148 plt7 = plot(layout=(2, 3), size=(1200, 800))
149 # Верхний ряд
150 plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=:blue, linewidth=2, title="Восприимчивые")
151 plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=:red, linewidth=2, title="Зараженные")
152 plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=:green, linewidth=2, title="Выздоровевшие")
153 # Нижний ряд
154 plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=:red, linewidth=2,
155 yscale=:log10, title="Лог. масштаб")
156 plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=:blue, linewidth=2,
157 title="Фазовый портрет", xlabel=L"S", ylabel=L"I")
158 plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, linewidth=2,
159 title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
160 # Сохранение графиков
161 savefig(plt1, plotsdir(script_name, "sir_main.png"))
162 savefig(plt2, plotsdir(script_name, "sir_infected.png"))
163 savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
164 savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
165 savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
166 savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
167 savefig(plt7, plotsdir(script_name, "sir_panel.png"))
168 # Бенчмарк для оценки производительности
169 println("\nБенчмарк решения:")
170 @benchmark solve(prob_ode, dt = 5t)
171 # Дополнительный анализ
172 println("\n== АНАЛИЗ РЕЗУЛЬТАТОВ ==")
173 println("Общая численность популяции (контроль): N = ", round(df_ode.N[1], digits=1))
174 println("Пиковое число зараженных: I_max = ", round(peak_value, digits=1))
175 println("Время достижения пика: t_peak = ", round(peak_time, digits=1), " дней")
176 println("Итоговое число переболевших: R(∞) = ", round(df_ode.R[end], digits=1))
177 println("Доля переболевших: ", round(df_ode.R[end]/df_ode.N[1]*100, digits=1), "%")
178 if R0 > 1
179 println("\nТеоретический анализ:")
180 println(" - Порог коллективного иммунитета: ", round((1-1/R0)*100, digits=1), "%")
181 println(" - Теоретический пик при S/N = 1/R0 = ", round(1/R0, digits=3))
182 end
```

Рисунок 2: *file*

```
[b964fa9f] + LaTeXStrings v1.4.0
No Changes to `~/project/Manifest.toml`
srluipp@vbox:~/project$ julia --project scripts/sir_ode.jl
Параметры модели SIR:
β (вероятность заражения) = 0.05
с (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R0 = с * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 = 0.0

Бенчмарк решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
- Порог коллективного иммунитета: 50.0%
- Теоретический пик при S/N = 1/R0 = 0.5
srluipp@vbox:~/project$
```

Рисунок 3: output

В plots оказались следующие графики: динамика эффективного репродуктивного числа, которая падает к 40 дню (рис. 4).

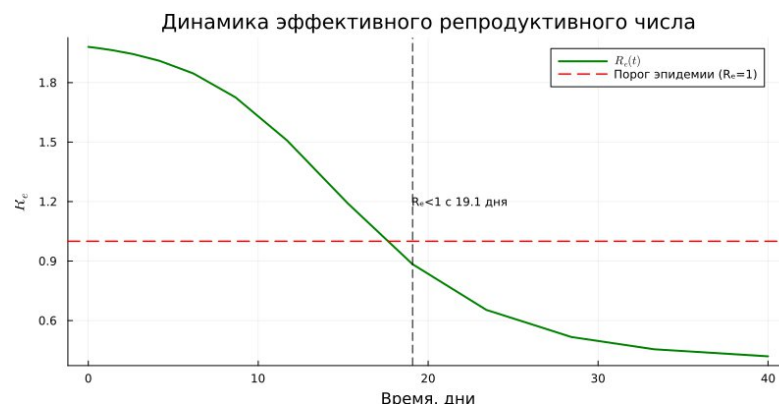


Рисунок 4: dinamic

Динамика числа зараженных, где пик всех зараженных приходится на 19 день (рис. 5).

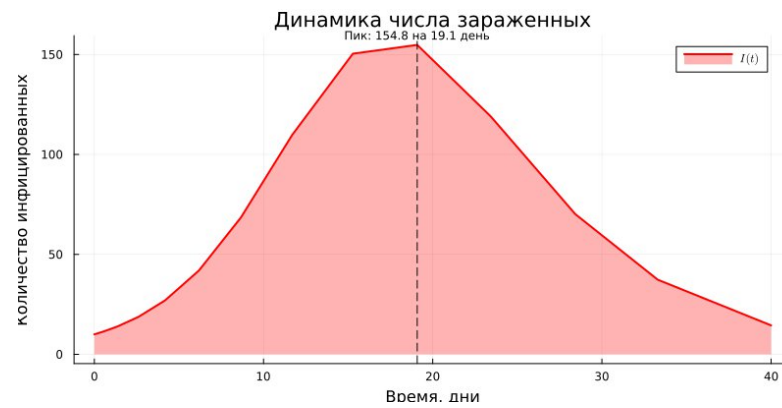


Рисунок 5: *infecteddynamic*

Следующий график - лог-шкала экспоненциального роста, и сама модель sir, показывающая динамику восприимчивых, инфицированных и выздоровевших (рис. 6) (рис. 7).

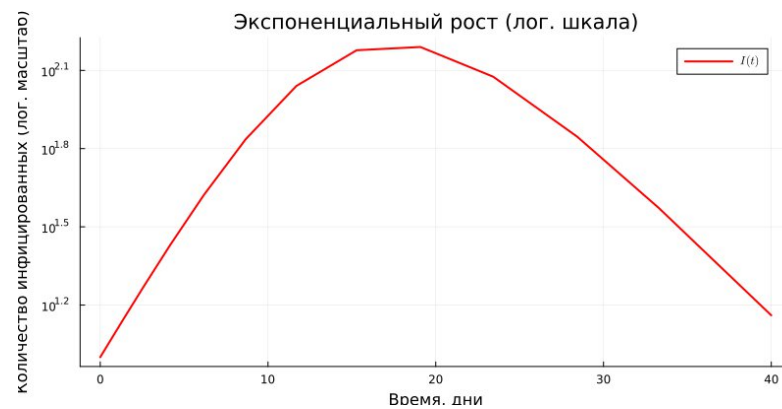


Рисунок 6: *exponentialgrowth*

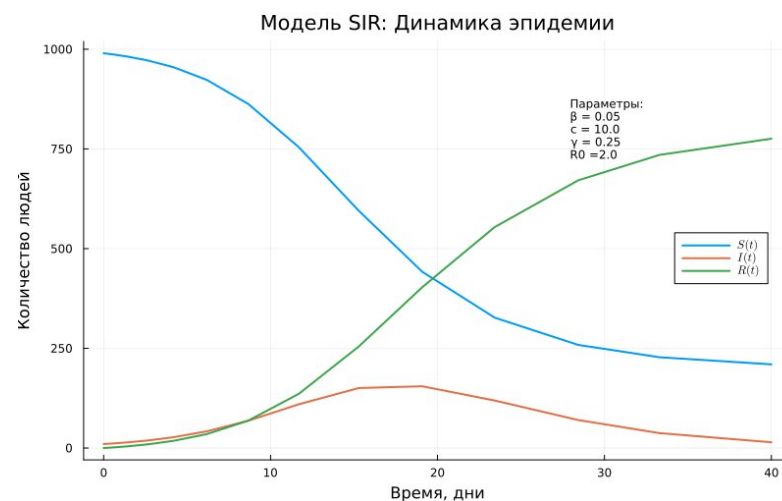


Рисунок 7: *sirmodel*

Далее выводятся все графики для каждой из групп по отдельности (рис. 8).

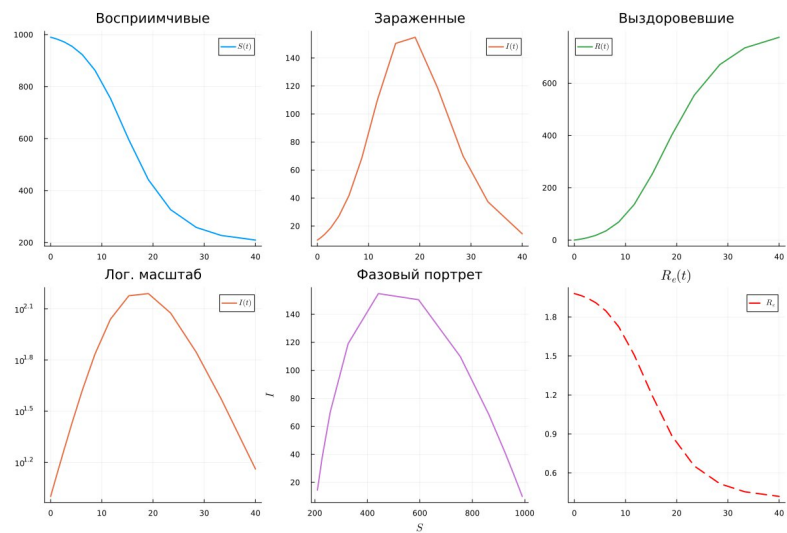


Рисунок 8: *grafs*

Показательна динамика эпидемии в процентах с порогом коллективного иммунитета в 50% (рис. 9).

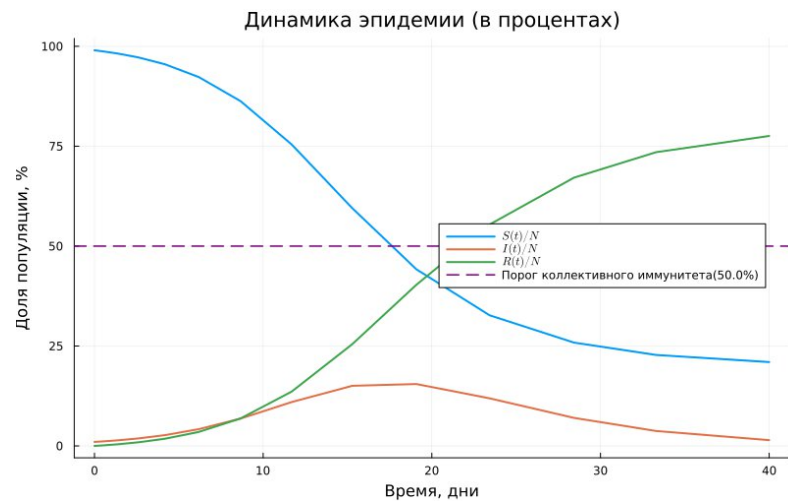


Рисунок 9: *percentage*

Завершаем анализ фазовым портретом модели sir(рис. 10) .

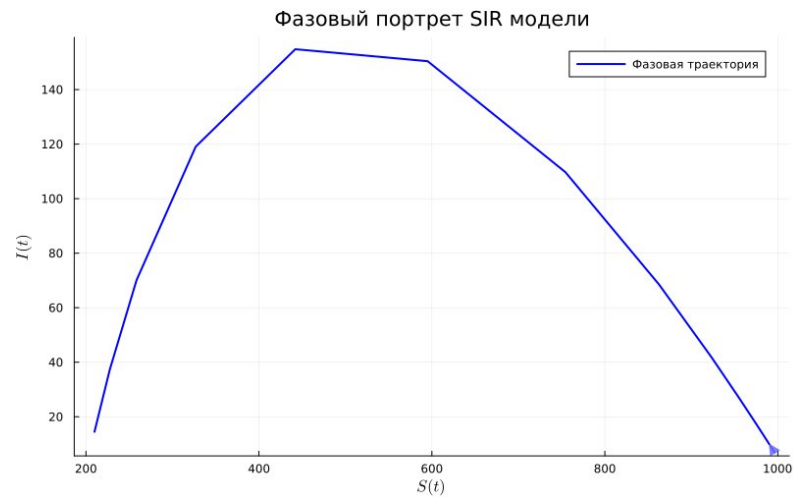


Рисунок 10: phaseportrait

Создаем новый файл `lv_ode` для модели Лотки–Вольтерры, описывающей динамику взаимодействия двух видов: хищников и жертв (рис. 11).

```

223 peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv.t)
224 peak_time_predator, peak_value_prey = find_first_peak(df_lv.predator, df_lv.t)
225 if !isnan(peak_time_prey) && !isnan(peak_time_predator)
226 phase_shift = peak_time_predator - peak_time_prey
227 println("\nАнализ колебаний:")
228 println("Первый пик жертв: время = ", round(peak_time_prey, digits=2),
229 ", значение = ", round(peak_value_prey, digits=2))
230 println("Первый пик хищников: время = ", round(peak_time_predator, digits=2),
231 ", значение = ", round(peak_value_predator, digits=2))
232 println("Сдвиг фаз (хищники отстают): ", round(phase_shift, digits=2))
233 end
234 # Сохранение графиков
235 savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
236 savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
237 savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
238 savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
239 savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
240 savefig(plt6, plotsdir(script_name, "lv_panel.png"))
241 # Дополнительный анализ: чувствительность к параметрам
242 println("\n\n" * "="^60)
243 println("Анализ чувствительности")
244 println("="^60)
245 # Функция для анализа влияния параметров
246 function analyze_parameter_sensitivity(param_index, values, param_name)
247 println("\nАнализ чувствительности к параметру: ", param_name)
248 results = []
249 for val in values
250 p_test = copy(p_lv)
251 p_test[param_index] = val
252 prob_test = ODEProblem(lotka_voltterra!, u0_lv, tspan_lv, p_test)
253 sol_test = solve(prob_test, dt = dt_lv)
254 prey_end = sol_test.u[end][1]
255 predator_end = sol_test.u[end][2]
256 push!(results, (param=val, prey=prey_end, predator=predator_end))
257 println(" $(param_name)=$(val): жертвы=$(round(prey_end,2)), хищники=$(round(predator_end,2))")
258 end
259 return results
260 end
261 # Анализ чувствительности к ключевым параметрам
262 if false # Установите в true для выполнения анализа
263 println("\n1. Влияние скорости размножения жертв (a):")
264 analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "a")
265 println("\n2. Влияние смертности хищников (γ):")
266 analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "γ")
267 end
268 println("\nМоделирование завершено успешно!")

```



```

Период колебаний жертв: 4.0 единиц времени

=====
Анализ результатов
=====

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8

[ Warning: No strict ticks found
@ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
[ Warning: No strict ticks found
@ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
[ Warning: Invalid negative or zero value 0.0 found at series inde
@ Plots ~/.julia/packages/Plots/AE0LB/src/utis.jl:105
[ Warning: Invalid negative or zero value 0.0 found at series inde
@ Plots ~/.julia/packages/Plots/AE0LB/src/utis.jl:105
[ Warning: No strict ticks found
@ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194

```

Рисунок 13: *secondoutput*

В plots находим следующие графики: производные популяций (рис. 14).

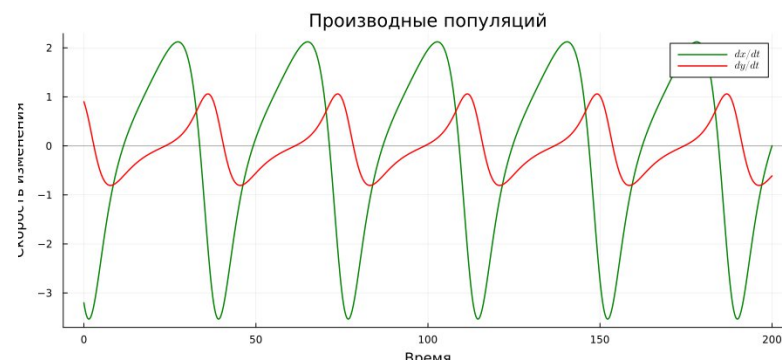


Рисунок 14: *population*

Динамику популяций жертв и хищников (рис. 15).

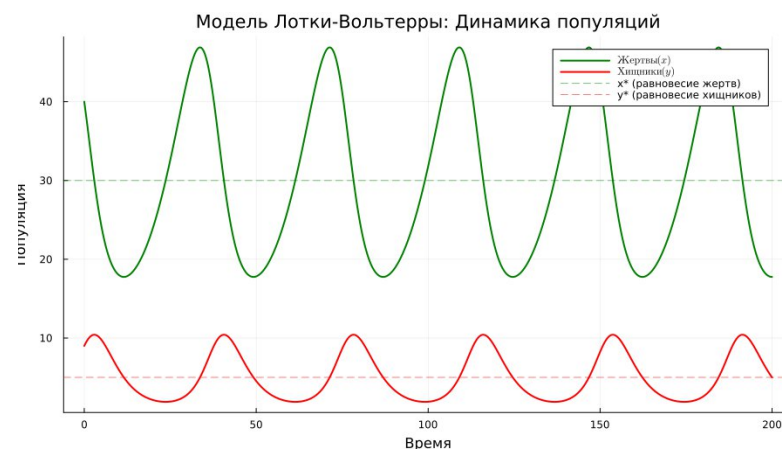


Рисунок 15: *dynamicpopulation*

Общий график для каждой популяции, фазовый портрет, скорость изменения и относительные изменения(рис. 16)

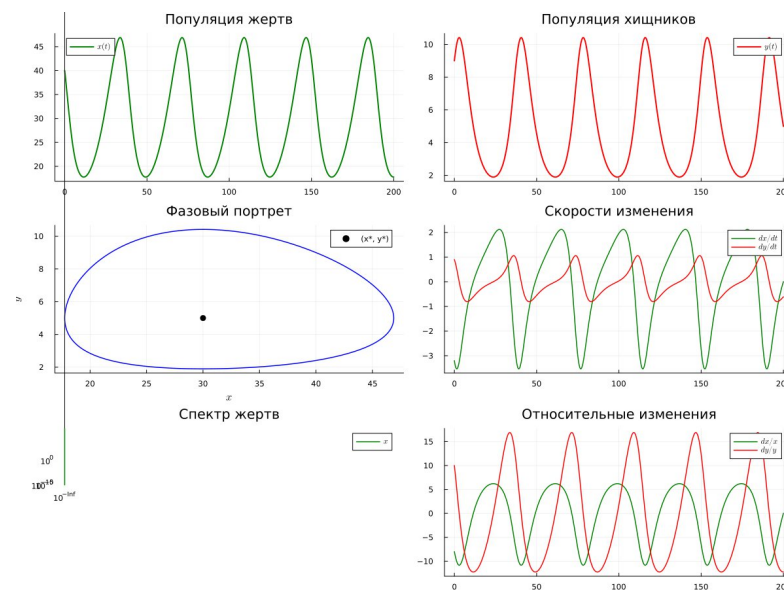


Рисунок 16: *bigscheme*

Фазовый портрет системы со стационарной точкой в 30 жертв и 5 хищников (рис. 17)

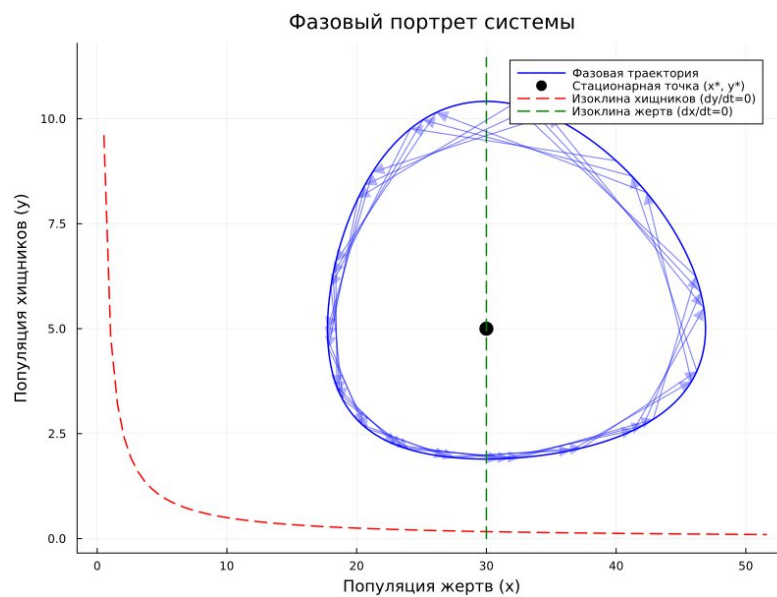


Рисунок 17: *phase*

Относительные темпы роста по времени (рис. 18).

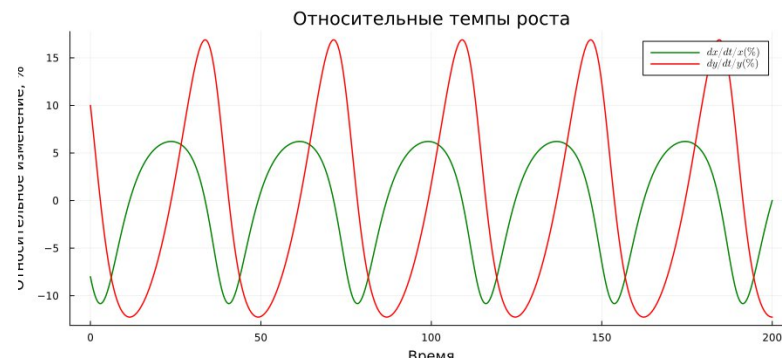


Рисунок 18: growth

И спектральный анализ Фурье (рис. 19)

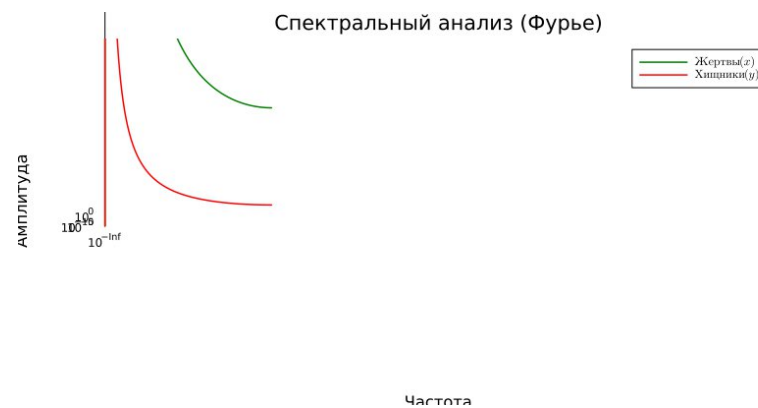


Рисунок 19: fyrie

Создаю tangle файл для форматов quarto и jupyter notebook (рис. 20).

```

Моделирование завершено успешно!
srluipp@vbox:~/project$ julia --project=. scripts/tangle.jl scripts/lv_ode.jl
Генерация из: scripts/lv_ode.jl
[ Info: generating plain script file from `~/project/scripts/lv_ode.jl`
[ Info: writing result to `~/project/scripts/lv_ode/lv_ode.jl`
✓ Чистый скрипт: /home/srluipp/project/scripts/lv_ode/lv_ode.jl
[ Info: generating markdown page from `~/project/scripts/lv_ode.jl`
[ Info: writing result to `~/project/markdown/lv_ode/lv_ode.qmd`
✓ Quarto: /home/srluipp/project/markdown/lv_ode/lv_ode.qmd
[ Info: generating notebook from `~/project/scripts/lv_ode.jl`
[ Info: writing result to `~/project/notebooks/lv_ode/lv_ode.ipynb`
✓ Notebook: /home/srluipp/project/notebooks/lv_ode/lv_ode.ipynb

Готово! Все файлы созданы.
srluipp@vbox:~/project$

```

Рисунок 20: tangle

Открываю рабочий лист в jupyter notebook lv_ode(рис. 21).

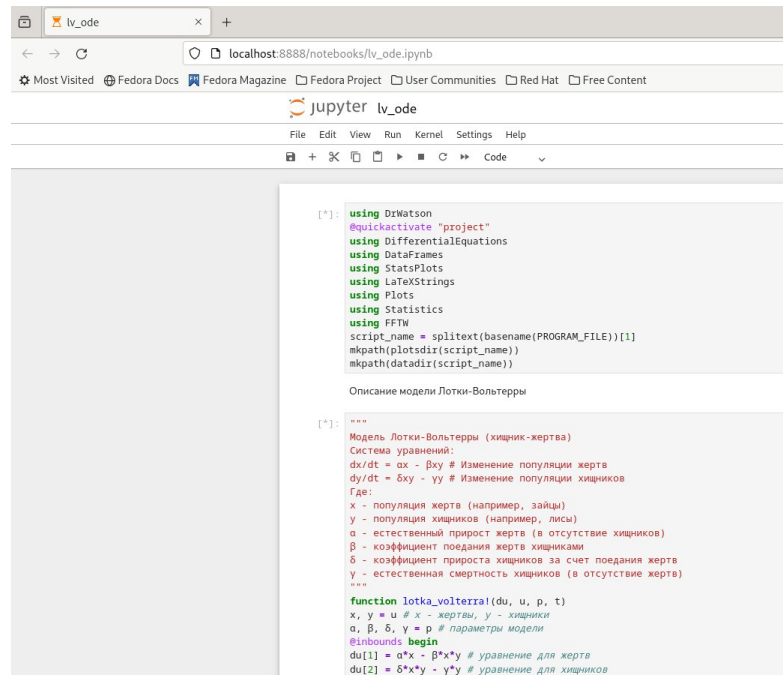


Рисунок 21: notebook

Некоторые выводы в блокноте jupyter notebook (рис. 22) (рис. 23) (рис. 24) (рис. 25) (рис. 26).

```
[7]: df_lv[1, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .* df_lv.predator
df_lv[1, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .- p_lv[4] .* df_lv.predator

[7]: 2001-element Vector{Float64}:
 0.9000000000000004
 0.8796081183812872
 0.8580494468233599
 0.835352334488115
 0.8115489002365717
 0.7866749261310106
 0.7607697060793308
 0.7338758893000268
 0.706039294083721
 0.6773087043294623
 0.6477350516317538
 0.6173741696029542
 0.5862805551332175
 ⋮
 -0.6917903984489653
 -0.6848226585569628
 -0.6776871951392999
 -0.6706369809304631
 -0.6634355041663743
 -0.6561720201747868
 -0.6488539394852814
 -0.641488358865548
 -0.6340828722589693
 -0.6266415740257927
 -0.6191738642026256
 -0.6116824631058659

Вывод информации о модели
```

Рисунок 22: notebookout

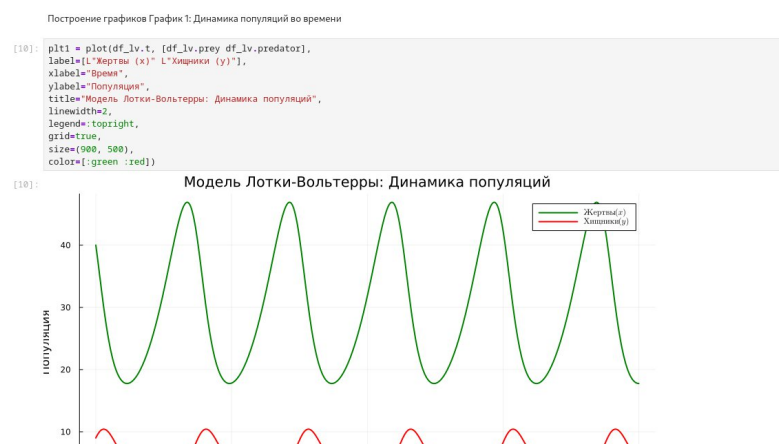


Рисунок 23: notebookj

```
println("\nОсновные статистики:")
println("Жерты: min = ", round(minimum(df_lv prey), digits=2),
", max = ", round(maximum(df_lv prey), digits=2),
", mean = ", round(mean(df_lv prey), digits=2))
println("Хищники: min = ", round(minimum(df_lv predator), digits=2),
", max = ", round(maximum(df_lv predator), digits=2),
", mean = ", round(mean(df_lv predator), digits=2))

=====
Анализ результатов
=====

Основные статистики:
Жерты: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Упрощенный анализ колебаний без поиска максимумов. Вместо сложного анализа сдвига фаз, просто посчитаем основные характеристики. Находим время первого
лика жертв (простой алгоритм)
```

Рисунок 24: jupyternotebook

```
srliupp@vbox:~/project — jupyter-notebook
To access the server, open this file in a browser:
file:///home/srliupp/.local/share/jupyter/runtime/jpserver-46790-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=37bf582976b198ccc2a18169e3b0f65e1d9d19d0a11dcf1a
http://127.0.0.1:8888/tree?token=37bf582976b198ccc2a18169e3b0f65e1d9d19d0a11dcf1a
[I 2026-02-22 17:43:01.934 ServerApp] Skipped non-installed server(s): basedpyright, bash-language-server, docker
file-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyref
ly, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript
-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-
json-languageserver-bin, yamll-language-server
[I 2026-02-22 17:43:05.436 JupyterNotebookApp] 302 GET /tree/lv_ode.ipynb?token=[secret] (17d0494cd2044138b489efa
d17b2d9040127.0.0.1) 3.76ms
[W 2026-02-22 17:43:13.751 ServerApp] Notebook lv_ode.ipynb is not trusted
[I 2026-02-22 17:43:15.481 ServerApp] Kernel started: 765f0e40-1d4d-4679-9190-1edd0efaf5a0
Starting kernel event loops.
[I 2026-02-22 17:43:17.326 ServerApp] Connecting to kernel 765f0e40-1d4d-4679-9190-1edd0efaf5a0.
[W 2026-02-22 17:43:17.366 ServerApp] The websocket_ping_timeout (90000) cannot be longer than the websocket_ping
interval (30000).
Setting websocket_ping_timeout=30000
[I 2026-02-22 17:43:17.386 ServerApp] Connecting to kernel 765f0e40-1d4d-4679-9190-1edd0efaf5a0.
[I 2026-02-22 17:43:17.529 ServerApp] Connecting to kernel 765f0e40-1d4d-4679-9190-1edd0efaf5a0.
[I 2026-02-22 17:45:14.619 ServerApp] Saving file at /lv_ode.ipynb
[I 2026-02-22 17:46:14.571 ServerApp] Saving file at /lv_ode.ipynb
```

Рисунок 25: jup

```
srliupp@vbox:~/project — jupyter-notebook
[Info] generating markdown page from '~/project/scripts/lv_ode.jl'
[Info] writing result to '~/project/markdown/lv_ode/lv_ode.qmd'
✓ Quarto: /home/srliupp/project/markdown/lv_ode/lv_ode.qmd
[Info] generating notebook from '~/project/scripts/lv_ode.jl'
[Info] writing result to '~/project/notebooks/lv_ode/lv_ode.ipynb'
✓ Notebook: /home/srliupp/project/notebooks/lv_ode/lv_ode.ipynb

Готово! Все файлы созданы.
srliupp@vbox:~/project$ notebooks/lv_ode/lv_ode.ipynb
bash: notebooks/lv_ode/lv_ode.ipynb: Permission denied
srliupp@vbox:~/project$ sudo notebooks/lv_ode/lv_ode.ipynb
[sudo] password for srliupp:
sudo: notebooks/lv_ode/lv_ode.ipynb: command not found
srliupp@vbox:~/project$ jupyter notebook notebooks/lv_ode/lv_ode.ipynb
[I 2026-02-22 17:43:00.452 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-02-22 17:43:00.458 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2026-02-22 17:43:00.466 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-02-22 17:43:00.473 ServerApp] notebook | extension was successfully linked.
[I 2026-02-22 17:43:01.248 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-02-22 17:43:01.307 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-02-22 17:43:01.310 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-02-22 17:43:01.312 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2026-02-22 17:43:01.318 LabApp] JupyterLab extension loaded from /home/srliupp/.local/lib/python3.12/site-pack
ages/jupyterlab
```

Рисунок 26: jupiterterm

Также делаю формат tangle для файла sir_ode (рис. 27) (рис. 28).

```

srluipp@vbox:~/project$ julia --project=. scripts/tangle.jl scripts/sir_ode.jl
Генерация из: scripts/sir_ode.jl
[ Info: generating plain script file from `~/project/scripts/sir_ode.jl`
[ Info: writing result to `~/project/scripts/sir_ode/sir_ode.jl`
✓ Чистый скрипт: /home/srluipp/project/scripts/sir_ode/sir_ode.jl
[ Info: generating markdown page from `~/project/scripts/sir_ode.jl`
[ Info: writing result to `~/project/markdown/sir_ode/sir_ode.qmd`
✓ Quarto: /home/srluipp/project/markdown/sir_ode/sir_ode.qmd
[ Info: generating notebook from `~/project/scripts/sir_ode.jl`
[ Info: writing result to `~/project/notebooks/sir_ode/sir_ode.ipynb`
✓ Notebook: /home/srluipp/project/notebooks/sir_ode/sir_ode.ipynb

Готово! Все файлы созданы.
srluipp@vbox:~/project$

```

Рисунок 27: *tangle2*

```

srluipp@vbox:~/project$ cd notebooks
srluipp@vbox:~/project/notebooks$ jupyter notebook sir_ode/sir_ode.ipynb
[I 2026-02-22 18:36:31.357 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-02-22 18:36:31.364 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2026-02-22 18:36:31.372 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-02-22 18:36:31.381 ServerApp] notebook | extension was successfully linked.
[I 2026-02-22 18:36:32.187 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-02-22 18:36:32.252 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-02-22 18:36:32.256 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-02-22 18:36:32.257 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2026-02-22 18:36:32.263 LabApp] JupyterLab extension loaded from /home/srluipp/.local/lib/python3.12/site-packages/jupyterlab
[I 2026-02-22 18:36:32.264 LabApp] JupyterLab application directory is /home/srluipp/.local/share/jupyter/lab
[I 2026-02-22 18:36:32.265 LabApp] Extension Manager is 'pypi'.

```

Рисунок 28: *format*

Вывожу файл `sir_ode` в jupyter notebook (рис. 29).

The screenshot shows a Jupyter Notebook window titled 'sir_ode'. The address bar indicates the notebook is running on 'localhost:8888/notebooks/sir_ode.ipynb'. The notebook interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The main content area displays a Julia script with the following code:

```

[ ]: using DrWatson
      @quickactivate "project"
      using DifferentialEquations
      using SimpleDiffEq
      using Tables
      using DataFrames
      using StatsPlots
      using LaTeXStrings # Для красивого отображения формул на графах
      using Plots
      using BenchmarkTools
      script_name = splitext(basename(PROGRAM_FILE))[1]
      mkpath(plotsdir(script_name))
      mkpath(datadir(script_name))
      function sir_ode!(du, u, p, t)
        (S, I, R) = u
        (β, c, γ) = p
        N = S + I + R
        @inbounds begin
          du[1] = -β * c * I / N * S
          du[2] = β * c * I / N * S - γ * I
          du[3] = γ * I
        end
        nothing
      end

      Параметры модели

[ ]: δt = 0.1
      tmax = 40.0
      tspan = (0.0, tmax)
      u0 = [990.0, 10.0, 0.0] # S, I, R
      p = [0.05, 10.0, 0.25] # β, c, γ

      Расчет базового репродуктивного числа

```

Рисунок 29: *sirjupiter*

Добавила в преамбуле `preamble.tex` пакет `juliamono`. В каталоге отчёта в файл `_quarto.yml` включила поддержку кода `julia`. Добавила после описания проекта (рис. 30) (рис. 31) (рис. 32).

```
project:
  title: "report"
  output-dir: "_output"

standalone: true
self-contained: true

## Julia support
engine: julia
julia:
  exeflags: ["--project=../project"]

## Generic options
lang: ru-RU
number-sections: true
toc: true
toc-title: "Содержание"
toc-depth: 2
## Crossref customization
```

Рисунок 30: *julia*

```
\usepackage[indentfirst]
\usepackage{float}
\floatplacement{figure}{H}
\IfFileExists{nlex-otf.sty}{
  %% Full TeXlive
  \usepackage[
    % math,

RM={Scale=0.94},SS={Scale=0.94},SSon={Scale=0.94},II={Scale=MatchLowercase,FakeStrike-
tch=0.9},
  DefaultFeatures={Ligatures=Common}
]{nlex-otf}
% \usepackage[Scale=MatchUppercase]{juliamono}
}{
  %% TinyTeX
  \usepackage{libertine}
}
\usepackage{juliamono}
```

Рисунок 31: *juliamono*

```

(image/43.png){#fig-043 width=70%}

(image/44.png){#fig-044 width=70%}

{{< include ../project/markdown/ly_ode/ly_ode.qmd >}}

{{< include ../project/markdown/sir_ode/sir_ode.qmd >}}
|
# Выводы

В ходе лабораторной работы я изучила модель SIR, модель Лотки-Вольтерры, узнала ее
практическое применение, изучила трехпараметрическую модель и ее практическую
надежность.

# Список литературы{.unnumbered}

::: {#refs}
:::

```

Рисунок 32: quarto

```

using DrWatson
@quickactivate "project"
using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings # Для красивого отображения формул на графиках
using Plots
using BenchmarkTools
gr()
Plots.default(fmt=:png)
backend(:gr)
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
function sir_ode!(du, u, p, t)
  (S, I, R) = u
  (β, c, γ) = p
  N = S + I + R
  @inbounds begin
    du[1] = -β * c * I / N * S
    du[2] = β * c * I / N * S - γ * I
    du[3] = γ * I
  end
  nothing
end

sir_ode! (generic function with 1 method)

```

Параметры модели

```

δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)

```



```
u0 = [990.0, 10.0, 0.0] # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 
```

```
3-element Vector{Float64}:
 0.05
 10.0
 0.25
```

Расчет базового репродуктивного числа

```
R0 = (p[2] * p[1]) / p[3] #  $R_0 = (c * \beta) / \gamma$ 
```

2.0

Создание и решение задачи

```
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt =  $\delta t$ )
```

retcode: Success

Interpolation: 3rd order Hermite

t: 15-element Vector{Float64}:

```
0.0
0.1
0.5331242537191838
1.3919008491040001
2.6102120755226386
4.1811820758323055
6.180123810566044
8.664951930903612
11.691425331813486
15.279370667110317
19.082963490744028
23.423425193442526
28.42949373098617
33.298779173639595
40.0
```

u: 15-element Vector{Vector{Float64}}:

```
[990.0, 10.0, 0.0]
[989.4990153200853, 10.247898072276143, 0.25308660763853114]
[987.1852964117763, 11.391111426852984, 1.4235921613706284]
[981.8312125227972, 14.026022042698543, 4.142765434504278]
[972.1392310589528, 18.757815561654184, 9.102953379393014]
[954.9905500491583, 27.007702518267596, 18.001747432574035]
[923.0706717929863, 41.9297677983272, 34.999560408686385]
[862.6705810342163, 68.49345419063425, 68.83596477514936]
[754.3022993851441, 109.7418618540929, 135.95583876076284]
[595.1014268847738, 150.4136326448068, 254.48494047041933]
[442.01411791013015, 154.80694988491862, 403.1789322049512]
[326.89511674939587, 119.06860394684871, 554.0362793037554]
[258.50852421889647, 70.1105219475306, 671.3809538335728]
```

```
[227.58338337085723, 37.32857063157995, 735.0880459975627]  
[209.8412140201436, 14.473644966455813, 775.6851410134005]
```

Подготовка данных в DataFrame

```
df_ode = DataFrame(Tables.table(sol_ode'))  
rename!(df_ode, ["S", "I", "R"])  
df_ode[:, :t] = sol_ode.t  
df_ode[:, :N] = df_ode.S + df_ode.I + df_ode.R
```

15-element Vector{Float64}:

```
1000.0  
1000.0  
999.9999999999999  
1000.0  
1000.0  
999.9999999999999  
1000.0  
999.9999999999999  
999.9999999999999  
1000.0  
1000.0  
1000.0  
999.9999999999999  
1000.0  
1000.0
```

Вывод параметров модели

```
println("Параметры модели SIR:")  
println("β (вероятность заражения) = ", p[1])  
println("с (среднее число контактов) = ", p[2])  
println("γ (скорость выздоровления) = ", p[3])  
println("R0 = с * β / γ = ", round(R0, digits=3))  
println("Средняя продолжительность болезни = ", round(1/p[3], digits=2), " дней")  
println("Начальные условия: S0 = ", u0[1], ", I0 = ", u0[2], ", R0 = ", u0[3])
```

Параметры модели SIR:

β (вероятность заражения) = 0.05
с (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R0 = с * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 = 0.0

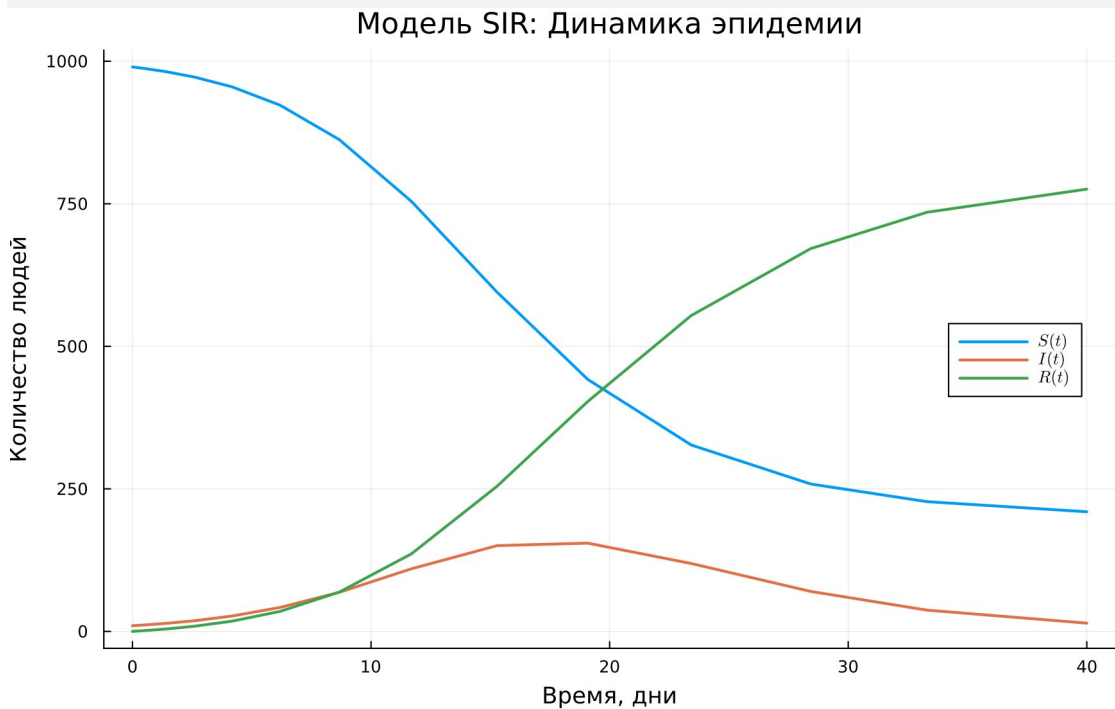
1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп

```
plt1 = @df df_ode plot(:t,  
[:S :I :R],  
label=[L"S(t)" L"I(t)" L"R(t)"],  
xlabel="Время, дни",  
ylabel="Количество людей",
```

```

title="Модель SIR: Динамика эпидемии",
linewidth=2,
legend=:right,
grid=true,
size=(800, 500))

```



Добавление аннотаций с параметрами

```

annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
text("Параметры:\n $\beta = $(p[1])$ \n $c = $(p[2])$ \n $\gamma = $(p[3])$ \n $R_0 = $(round(R0, digits=2))$ ",
8, :left))

```

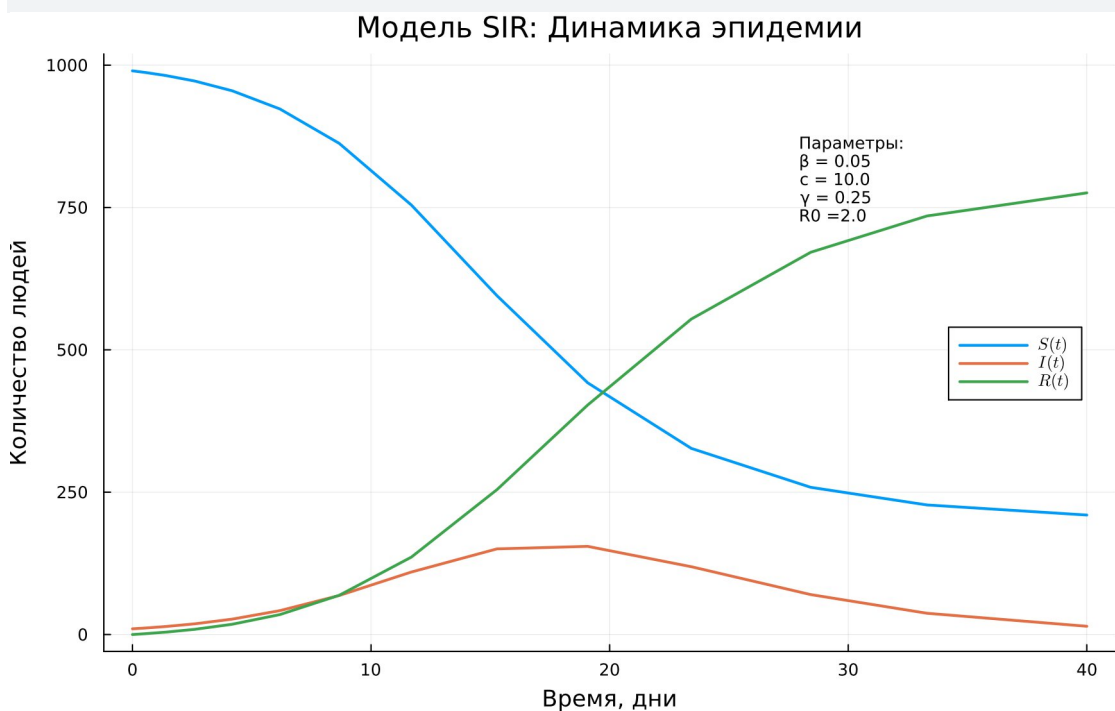


График только инфицированных (I)

```
plt2 = @df df_ode plot(:t, :I,
label=L"I(t)",
xlabel="Время, дни",
ylabel="Количество инфицированных",
title="Динамика числа зараженных",
color=:red,
linewidth=2,
fill=(0, 0.3, :red),
grid=true,
size=(800, 400))
```



Отметка пика эпидемии

```
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false, linewidth=1)
annotate!(plt2, peak_time, peak_value * 1.05,
text("Пик: $(round(peak_value, digits=1)) на $(round(peak_time, digits=1)) день", 8, :top))
```

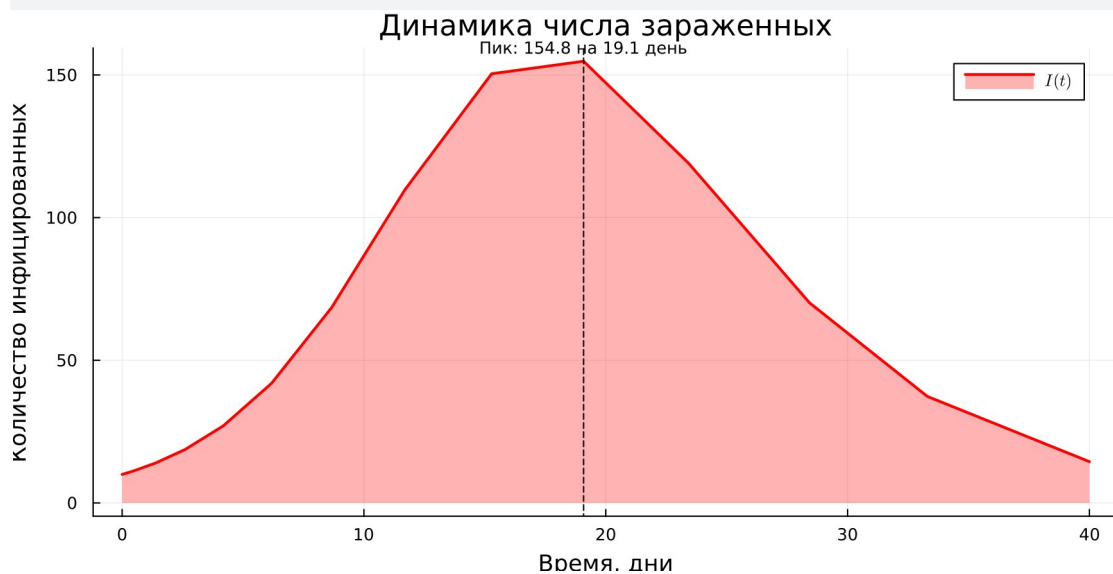


График в логарифмическом масштабе (для анализа экспоненциального роста)

```
plt3 = @df df_ode plot(:t, :I,  
label=L"I(t)",  
xlabel="Время, дни",  
ylabel="Количество инфицированных (лог. масштаб)",  
title="Экспоненциальный рост (лог. шкала)",  
yscale=:log10,  
color=:red,  
linewidth=2,  
grid=true,  
size=(800, 400))
```

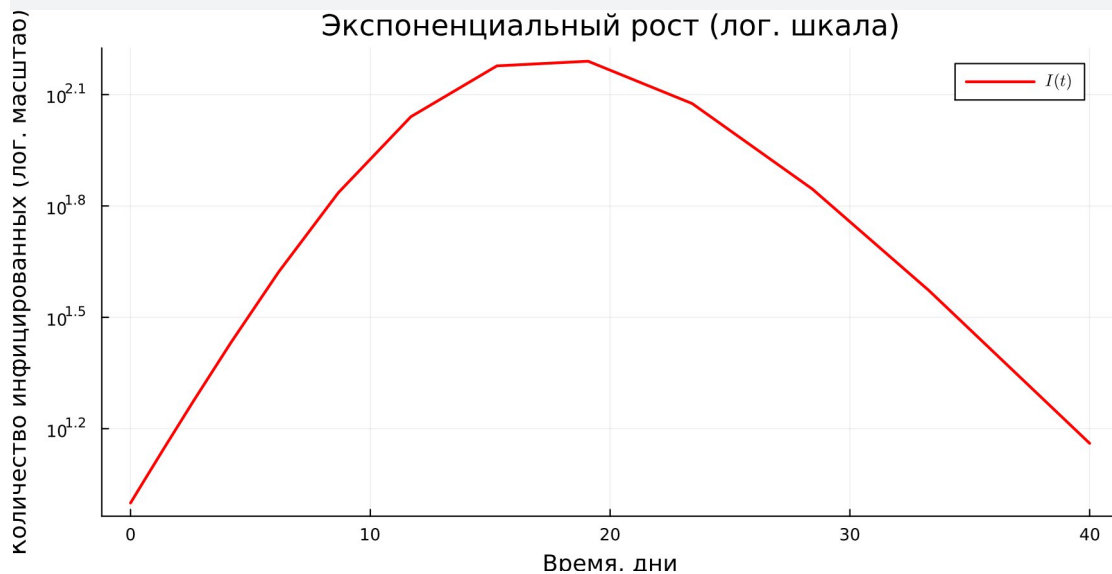
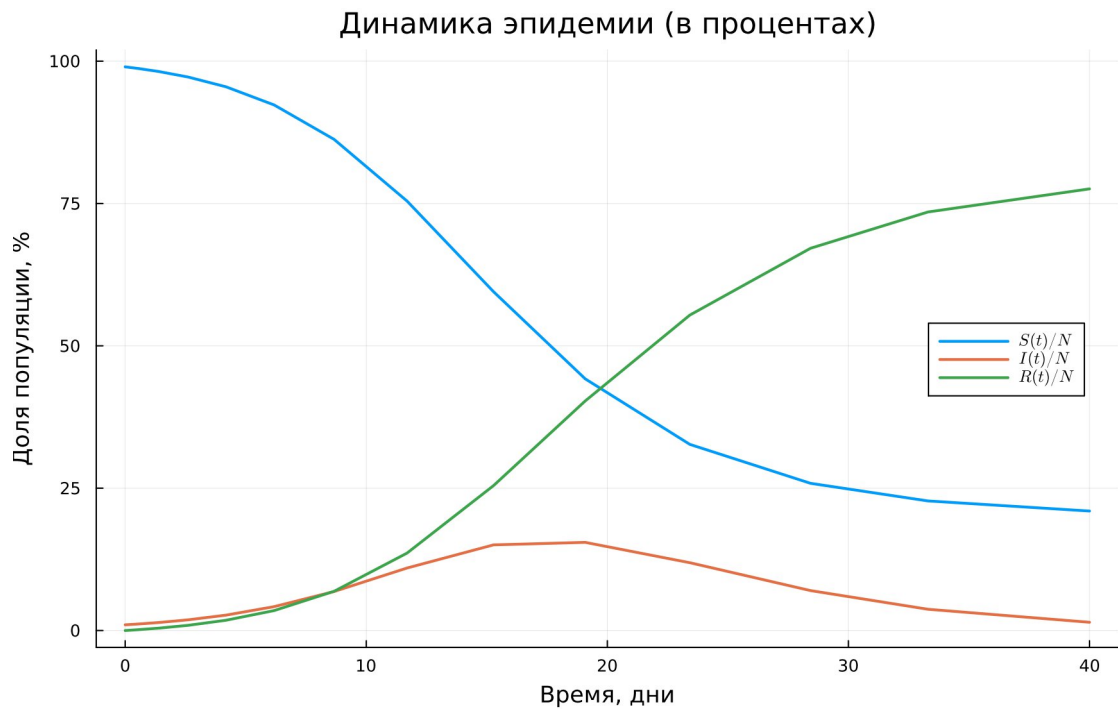


График долей населения (в процентах)

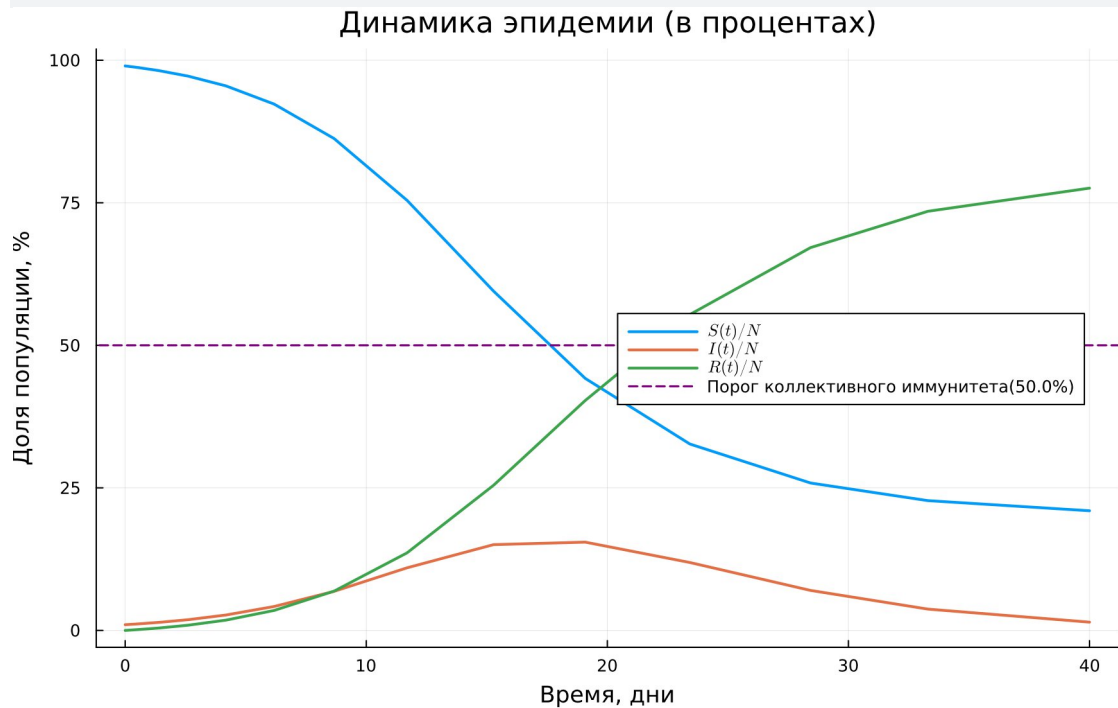
```
plt4 = @df df_ode plot(:t, [:S :I :R] ./ df_ode.N .* 100,  
label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],  
xlabel="Время, дни",  
ylabel="Доля популяции, %",  
title="Динамика эпидемии (в процентах)",  
linewidth=2,  
legend=:right,  
grid=true,  
size=(800, 500))
```



Горизонтальная линия для порога коллективного иммунитета

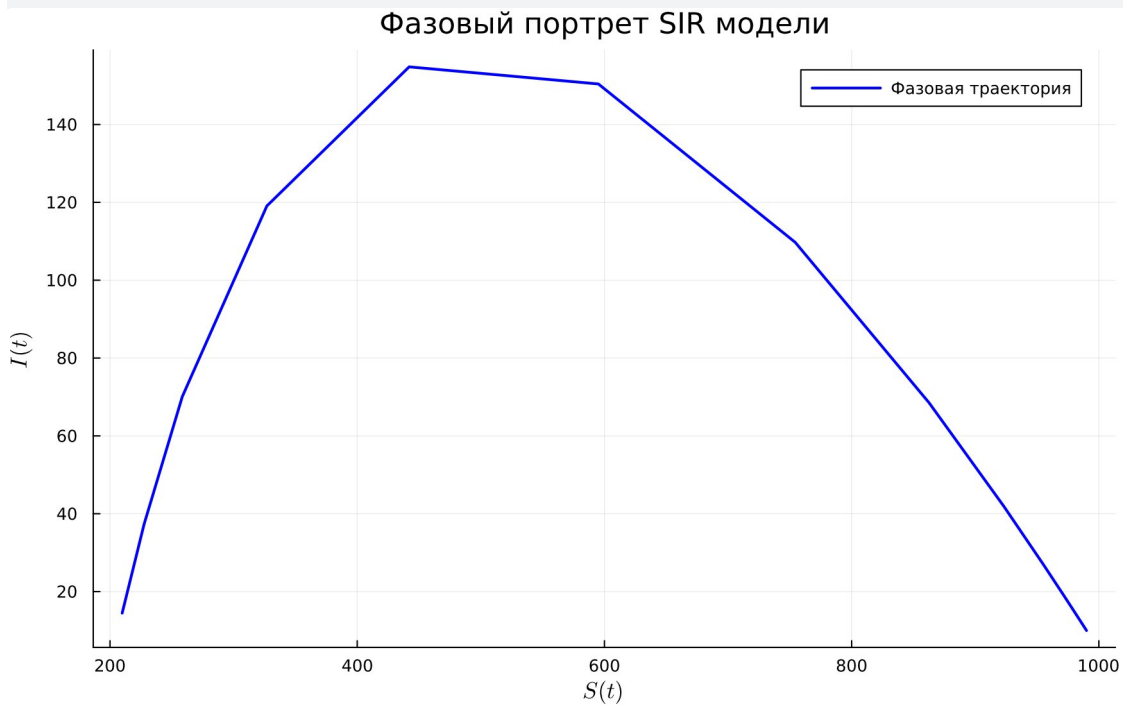
```

if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyle=:dash,
    label="Порог коллективного иммунитета($(round(herd_immunity_threshold,
    digits=1)))%",
    linewidth=1.5)
end
  
```



Фазовый портрет (I vs S)


```
plt5 = plot(df_ode.S, df_ode.I,
label="Фазовая траектория",
xlabel=L"S(t)",
ylabel=L"I(t)",
title="Фазовый портрет SIR модели",
color=:blue,
linewidth=2,
grid=true,
size=(800, 500),
legend=:topright)
```

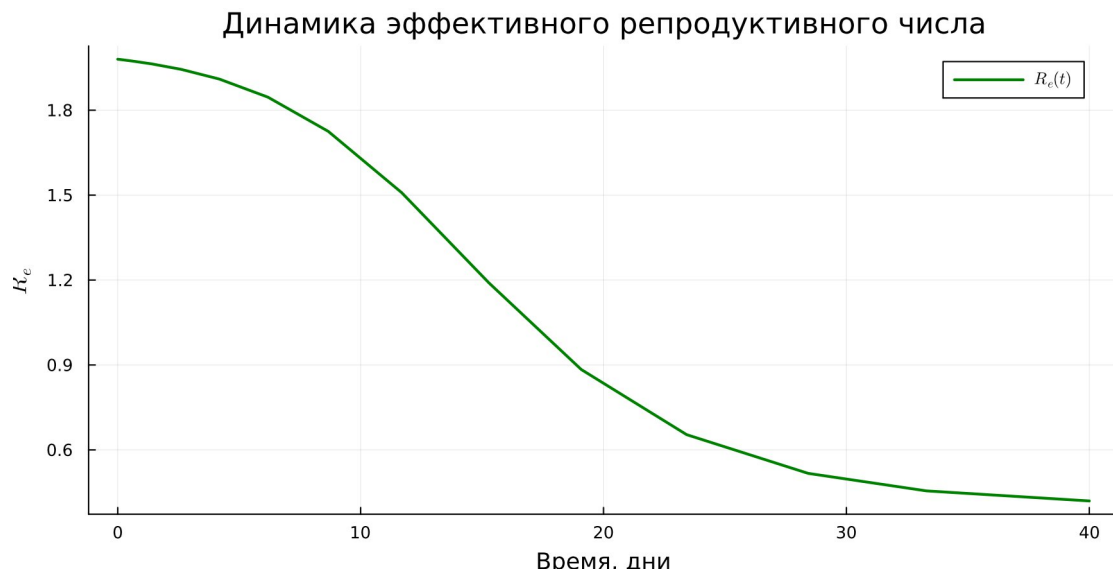


Добавление стрелок направления

```
for i in 1:50:length(df_ode.S)-1
plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.I[i+1]],
arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```

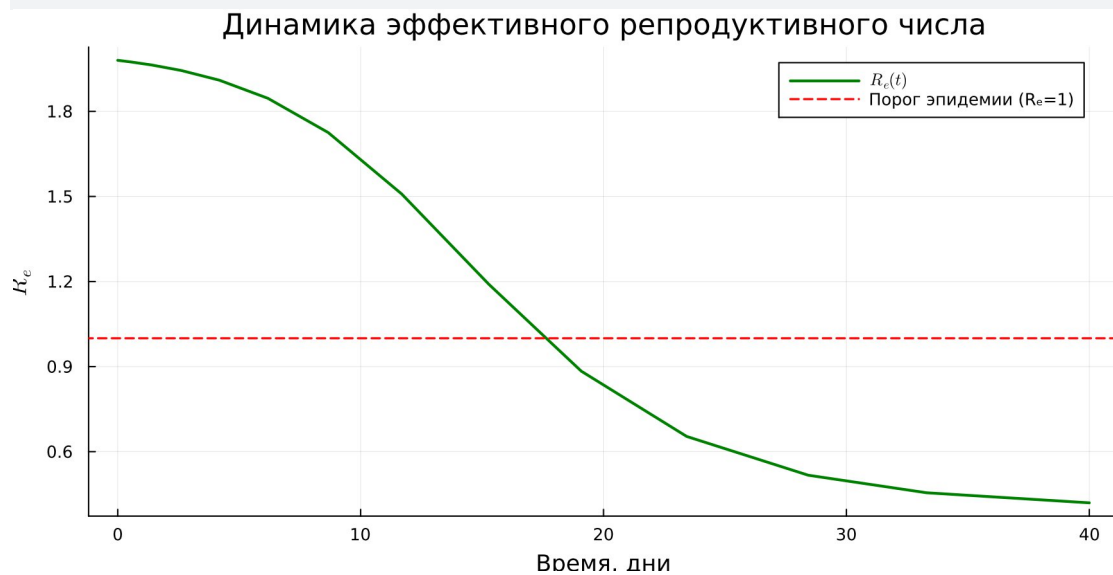
График R_e - эффективного репродуктивного числа

```
df_ode[:, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
label=L"R_e(t)",
xlabel="Время, дни",
ylabel=L"R_e",
title="Динамика эффективного репродуктивного числа",
color=:green,
linewidth=2,
grid=true,
size=(800, 400))
```



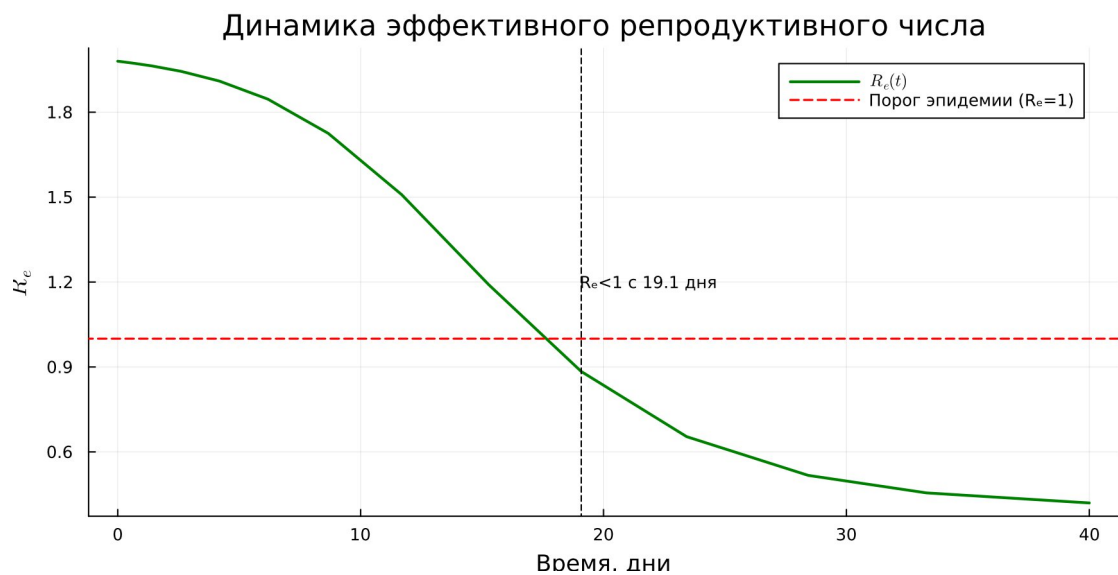
Горизонтальная линия на уровне 1

```
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="Порог эпидемии ( $R_e=1$ )", linewidth=1.5)
```



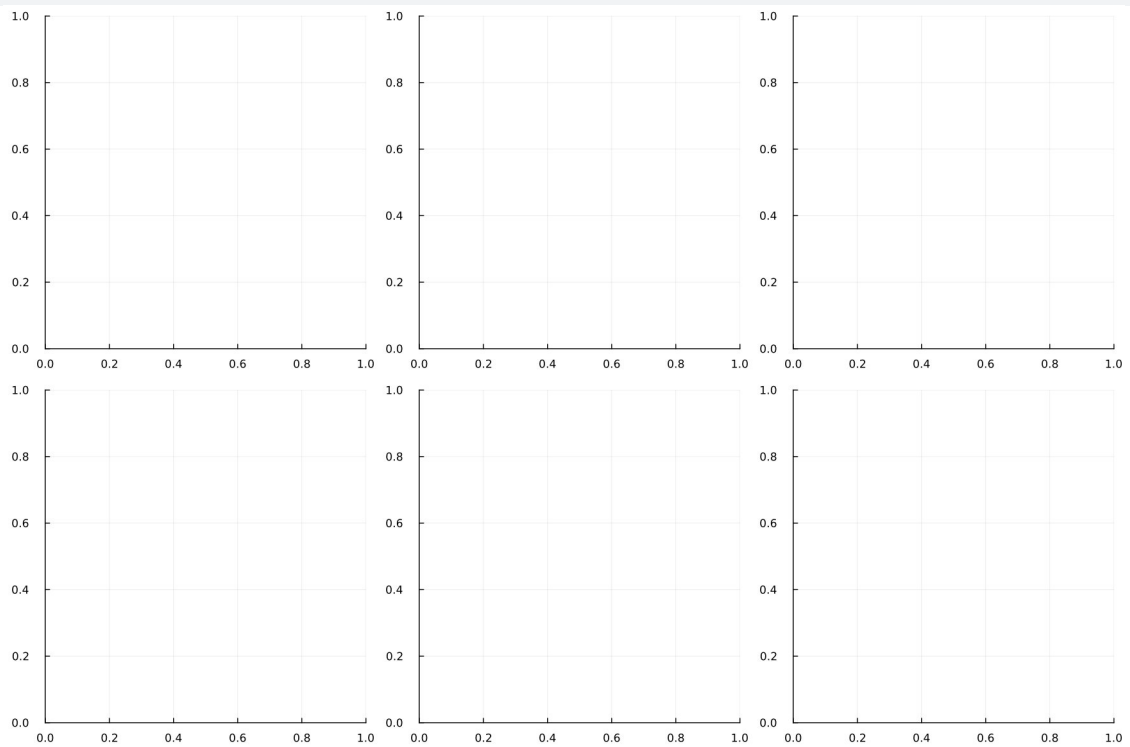
Отметка момента, когда R_e становится < 1

```
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=false, linewidth=1)
    annotate!(plt6, cross_time, 1.2,
    text(" $R_e < 1$  с $(round(cross_time, digits=1))$ дня", 8, :left))
end
```



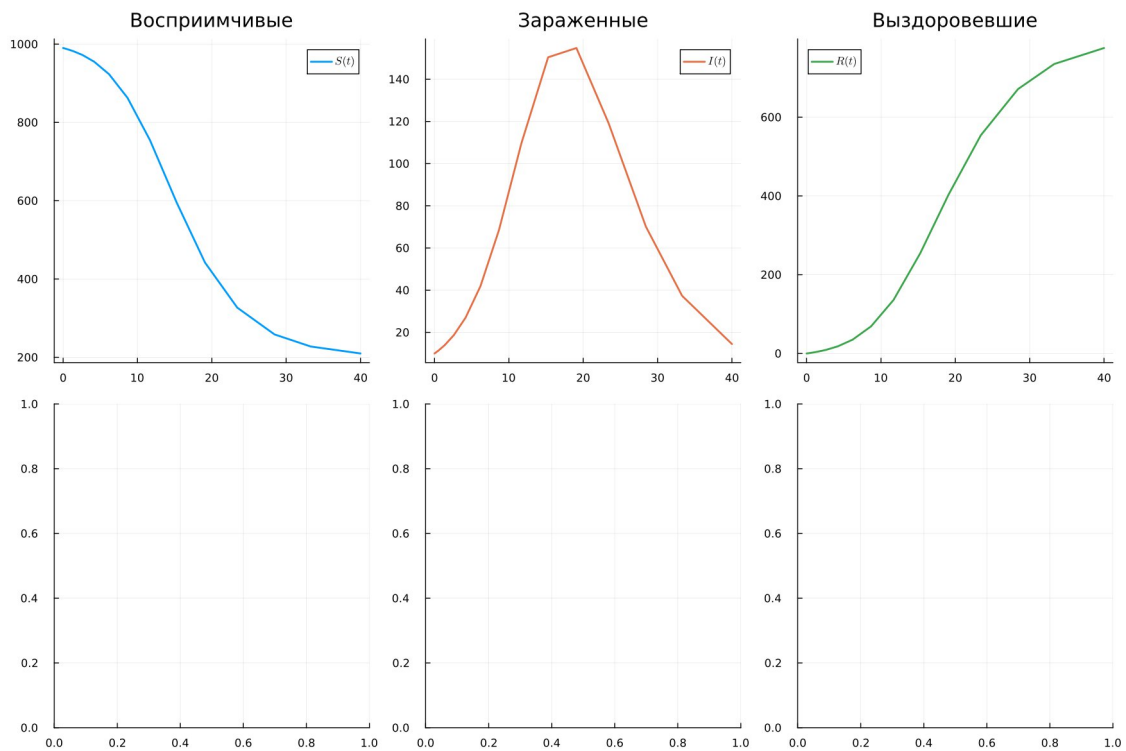
Компактный график всех кривых в одной панели

```
plt7 = plot(layout=(2, 3), size=(1200, 800))
```



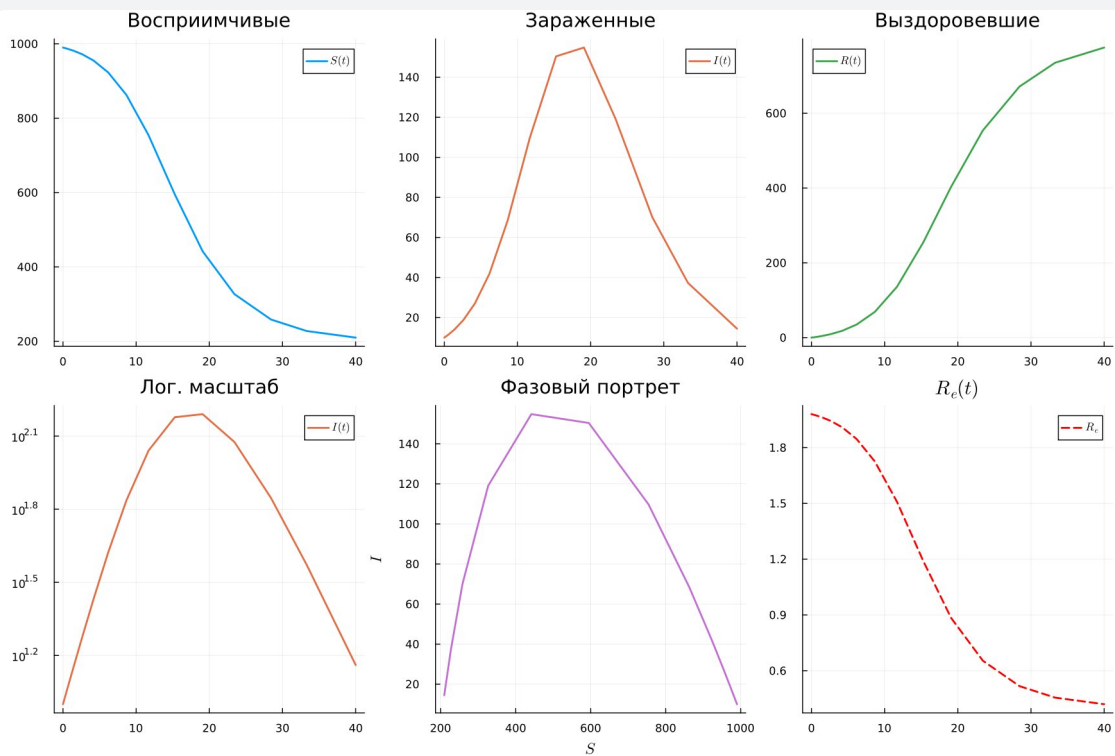
Верхний ряд

```
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidth=2, title="Восприимчивые")
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth=2, title="Зараженные")
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidth=2, title="Выздоровевшие")
```



Нижний ряд

```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=:red, linewidth=2,
yscale=:log10, title="Лог. масштаб")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=:blue, linewidth=2,
title="Фазовый портрет", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, linewidth=2,
title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
```



Сохранение графиков

```

savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))

```

"/home/srluipp/plots/sir_panel.png"

Бенчмарк для оценки производительности

```

println("\nБенчмарк решения:")
@benchmark solve(prob_ode, dt = dt)

```

Бенчмарк решения:

BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.

Range (min ... max): 61.684 μ s ... 19.493 ms | GC (min ... max): 0.00% ... 98.78%

Time (median): 66.118 μ s | GC (median): 0.00%

Time (mean $\pm$ σ): 79.653 μ s $\pm$ 257.863 μ s | GC (mean $\pm$ σ): 7.02% $\pm$ 2.71%



61.7 μ s Histogram: log(frequency) by time 176 μ s <

Memory estimate: 52.00 KiB, allocs estimate: 460.

Дополнительный анализ

```

println("\n=== АНАЛИЗ РЕЗУЛЬТАТОВ ===")
println("Общая численность популяции (контроль): N = ", round(df_ode.N[1], digits=1))
println("Пиковое число зараженных: I_max = ", round(peak_value, digits=1))
println("Время достижения пика: t_peak = ", round(peak_time, digits=1), " дней")
println("Итоговое число переболевших: R( $\infty$ ) = ", round(df_ode.R[end], digits=1))
println("Доля переболевших: ", round(df_ode.R[end]/df_ode.N[1]*100, digits=1), "%")
if R0 > 1
println("\nТеоретический анализ:")
println(" - Порог коллективного иммунитета: ", round((1-1/R0)*100, digits=1), "%")
println(" - Теоретический пик при S/N = 1/R0 = ", round(1/R0, digits=3))
end

```

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===

Общая численность популяции (контроль): N = 1000.0

Пиковое число зараженных: I_max = 154.8

Время достижения пика: t_peak = 19.1 дней

Итоговое число переболевших: R(∞) = 775.7

Доля переболевших: 77.6%

Теоретический анализ:

- Порог коллективного иммунитета: 50.0%
- Теоретический пик при $S/N = 1/R_0 = 0.5$

```
using DrWatson
@quickactivate "project"
using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW
script_name = splittest(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

"/home/srluipp/data"
```

Описание модели Лотки-Вольтерры

""""

Модель Лотки-Вольтерры (хищник-жертва)

Система уравнений:

$dx/dt = \alpha x - \beta xy$ # Изменение популяции жертв

$dy/dt = \delta xy - \gamma y$ # Изменение популяции хищников

Где:

x - популяция жертв (например, зайцы)

y - популяция хищников (например, лисы)

α - естественный прирост жертв (в отсутствие хищников)

β - коэффициент поедания жертв хищниками

δ - коэффициент прироста хищников за счет поедания жертв

γ - естественная смертность хищников (в отсутствие жертв)

""""

```
function lotka_volterra!(du, u, p, t)
```

```
x, y = u # x - жертвы, y - хищники
```

```
 $\alpha$ ,  $\beta$ ,  $\delta$ ,  $\gamma$  = p # параметры модели
```

```
@inbounds begin
```

```
du[1] =  $\alpha$ *x -  $\beta$ *x*y # уравнение для жертв
```

```
du[2] =  $\delta$ *x*y -  $\gamma$ *y # уравнение для хищников
```

```
end
```

```
nothing
```

```
end
```

```
p_lv = [0.1, #  $\alpha$ : скорость размножения жертв
```

```
0.02, #  $\beta$ : скорость поедания жертв хищниками
```

```
0.01, #  $\delta$ : коэффициент конверсии пищи (жертв) в хищников
```

```
0.3] #  $\gamma$ : смертность хищников
```

4-element Vector{Float64}:

0.1
0.02
0.01
0.3

Начальные условия: [жертвы, хищники]

u0_lv = [40.0, 9.0] # начальная популяция

2-element Vector{Float64}:

40.0
9.0

Временные параметры

tspan_lv = (0.0, 200.0) # длительность симуляции

dt_lv = 0.01 # шаг интегрирования

0.01

Создание и решение задачи

prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)

sol_lv = solve(prob_lv,

dt = dt_lv,

Tsit5(), # Метод 5-го порядка

reltol=1e-8, # Относительная точность

abstol=1e-10, # Абсолютная точность

saveat=0.1, # Сохраняем каждые 0.1 единицы времени

dense=true # Включаем плотный вывод для интерполяции

)

└ Warning: Verbosity toggle: dense_output_saveat

| Dense output is incompatible with saveat. Please use the SavingCallback from the
| Callback Library to mix the two behaviors.

└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/oyzWP/src/solve.jl:132

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 2001-element Vector{Float64}:

0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0


```

1.1
1.2
⋮
198.9
199.0
199.1
199.2
199.3
199.4
199.5
199.6
199.7
199.8
199.9
200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.67773201890384, 9.088990237207637]
 [39.35113789503168, 9.175882726306998]
 [39.02053587808482, 9.260562174341326]
 [38.68624820281499, 9.342916311942016]
 [38.34860018759226, 9.422836265415746]
 [38.007919320045765, 9.500216918706203]
 [37.664534369328166, 9.57495725033006]
 [37.31877446654823, 9.646960666854818]
 [36.970968217678404, 9.716135308317776]
 [36.62144287329975, 9.78239432138995]
 [36.27052336512251, 9.845656154267314]
 [35.91853163311434, 9.905844751305578]
 ⋮
 [17.889675634214175, 5.712401894068308]
 [17.865436242318786, 5.643570483722319]
 [17.8436734538774, 5.5754402198920205]
 [17.82435314492936, 5.5080193185068556]
 [17.807442037033802, 5.441315154551656]
 [17.79290768632072, 5.375334300040312]
 [17.780718473179366, 5.310082577777455]
 [17.770843588540256, 5.24556508481179]
 [17.76325301314867, 5.1817862454809696]
 [17.75791751745931, 5.118749811721094]
 [17.754808657966603, 5.05645887359248]
 [17.75389874795975, 4.994915937053494]

```

Подготовка данных

```

df_lv = DataFrame()
df_lv[:, :t] = sol_lv.t
df_lv[:, :prey] = [u[1] for u in sol_lv.u] # жертвы
df_lv[:, :predator] = [u[2] for u in sol_lv.u] # хищники

```

2001-element Vector{Float64}:

```
9.0
9.088990237207637
9.175882726306998
9.260562174341326
9.342916311942016
9.422836265415746
9.500216918706203
9.57495725033006
9.646960666854818
9.716135308317776
9.78239432138995
9.845656154267314
9.905844751305578
⋮
5.712401894068308
5.643570483722319
5.5754402198920205
5.5080193185068556
5.441315154551656
5.375334300040312
5.310082577777455
5.24556508481179
5.1817862454809696
5.118749811721094
5.05645887359248
4.994915937053494
```

Расчет производных для анализа

```
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .* df_lv.predator
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .- p_lv[4] .* df_lv.predator
```

2001-element Vector{Float64}:

```
0.90000000000000004
0.8796081183812872
0.8580494468233599
0.8353523334488115
0.8115489002365717
0.7866749261310106
0.7607697060793308
0.7338758893000268
0.706039294083721
0.6773087043294623
0.6477356516317538
0.6173741696029542
0.5862805551332175
⋮
-0.6917903984489653
```

-0.6848226585569628
-0.6777687195139299
-0.6706369809304631
-0.6634355041663743
-0.6561720201747868
-0.6488539394852814
-0.6414883588865548
-0.6340820722589693
-0.6266415740257927
-0.6191730642026256
-0.6116824631058659

Вывод информации о модели

```
println("="^60)
println("Модель Лотки-Вольтерры (хищник-жертва)")
println("="^60)
println("\nПараметры модели:")
println("α (скорость размножения жертв) = ", p_lv[1])
println("β (скорость поедания жертв) = ", p_lv[2])
println("δ (коэффициент конверсии) = ", p_lv[3])
println("γ (смертность хищников) = ", p_lv[4])
println("\nНачальные условия:")
println("Жертвы (x0) = ", u0_lv[1])
println("Хищники (y0) = ", u0_lv[2])
```

=====

Модель Лотки-Вольтерры (хищник-жертва)

=====

Параметры модели:

α (скорость размножения жертв) = 0.1

β (скорость поедания жертв) = 0.02

δ (коэффициент конверсии) = 0.01

γ (смертность хищников) = 0.3

Начальные условия:

Жертвы (x0) = 40.0

Хищники (y0) = 9.0

Стационарные точки (нулевые изоклины)

```
x_star = p_lv[4] / p_lv[3] # стационарная точка для жертв
y_star = p_lv[1] / p_lv[2] # стационарная точка для хищников
println("\nСтационарные точки (положения равновесия):")
println("x* =  $\gamma/\delta$  = ", round(x_star, digits=3))
println("y* =  $\alpha/\beta$  = ", round(y_star, digits=3))
```

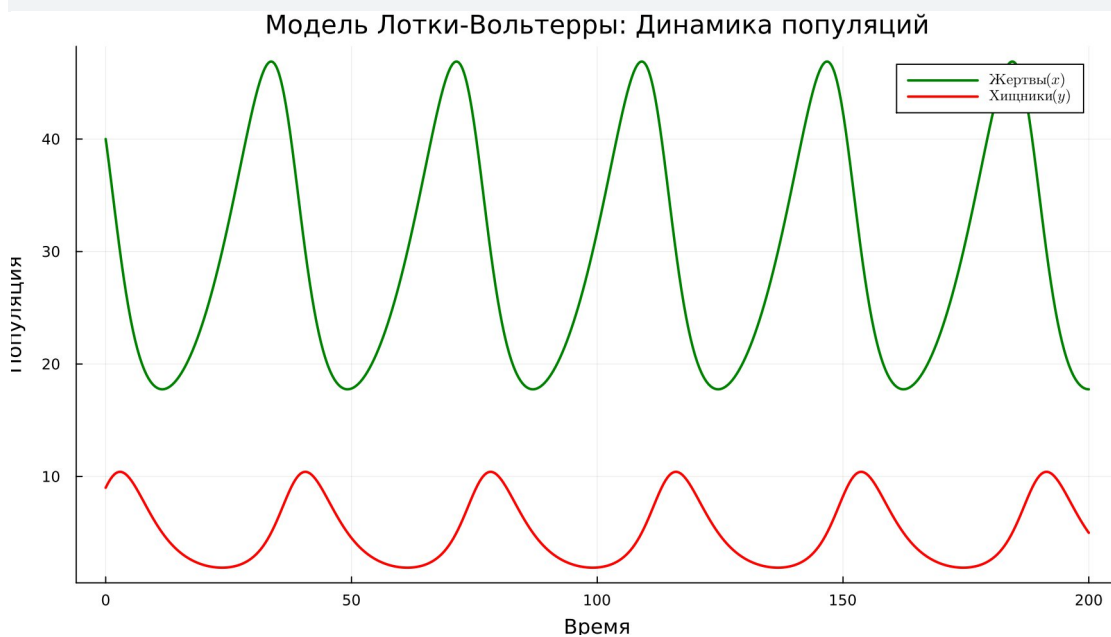
Стационарные точки (положения равновесия):

$$x^* = \gamma/\delta = 30.0$$

$$y^* = \alpha/\beta = 5.0$$

Построение графиков График 1: Динамика популяций во времени

```
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],  
label=[L"Жертвы (x)" L"Хищники (y)"],  
xlabel="Время",  
ylabel="Популяция",  
title="Модель Лотки-Вольтерры: Динамика популяций",  
linewidth=2,  
legend=:topright,  
grid=true,  
size=(900, 500),  
color=[:green :red])
```



Добавление стационарных уровней

```
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5, label="x* (равновесие  
жертв)")
```

```
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, label="y* (равновесие  
хищников)")
```

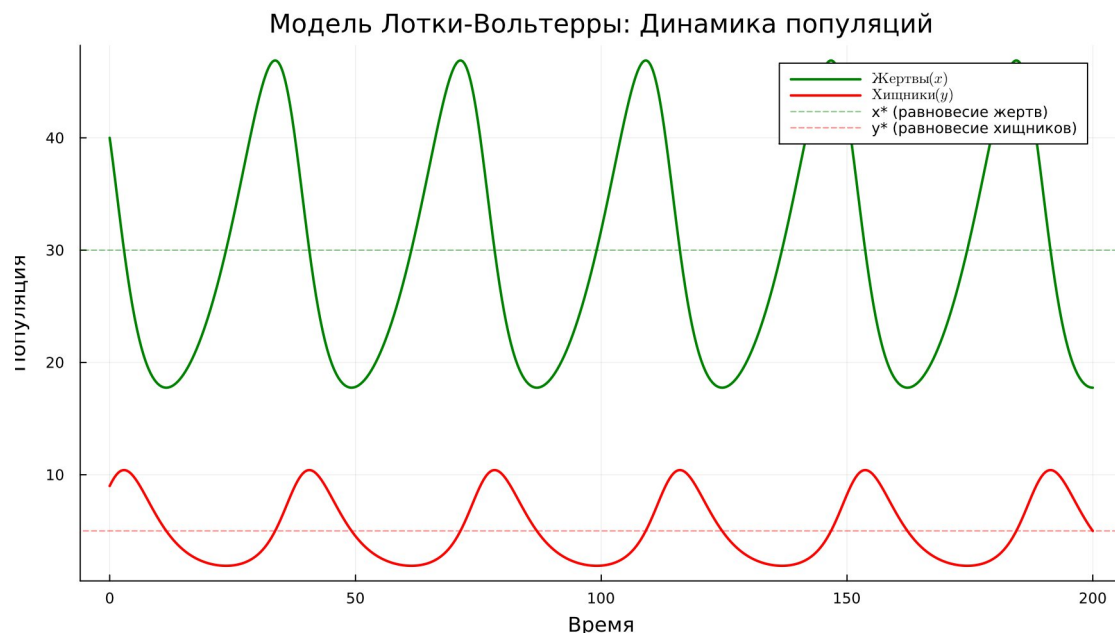
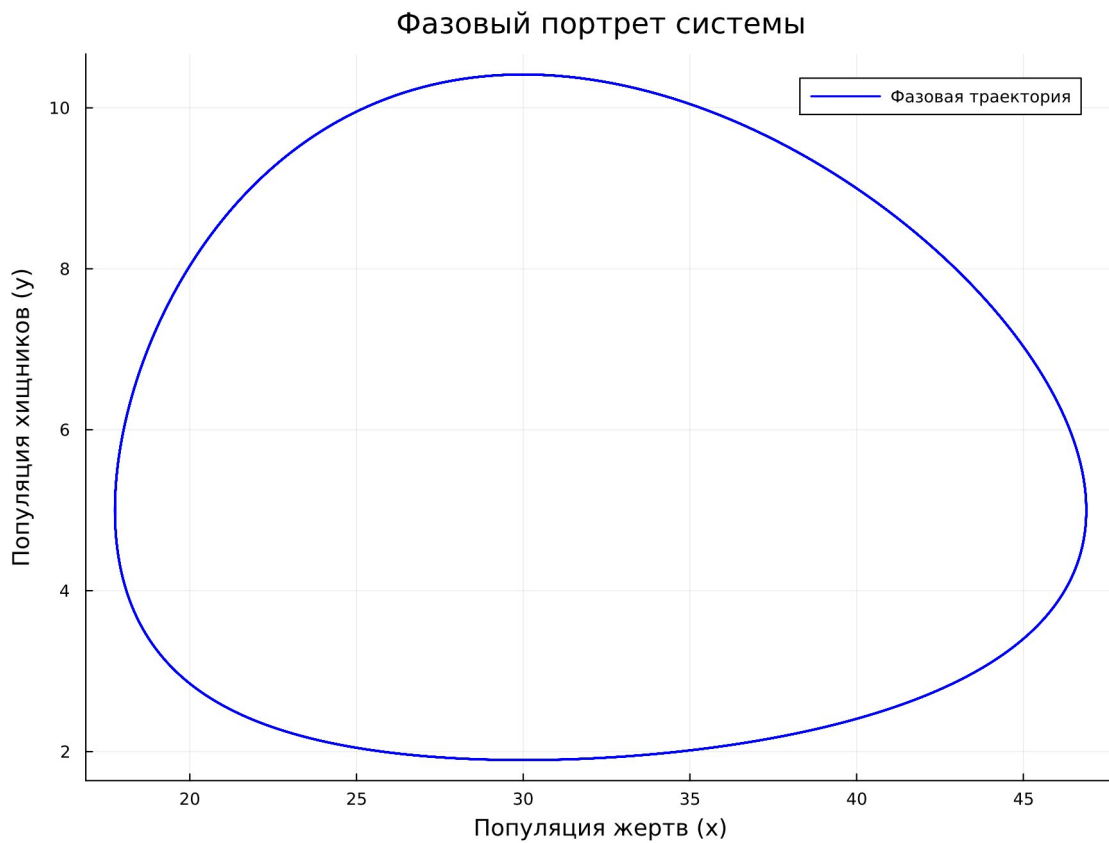


График 2: Фазовый портрет (хищники vs жертвы)

```
plt2 = plot(df_lv.prey, df_lv.predator,
label="Фазовая траектория",
xlabel="Популяция жертв (x)",
ylabel="Популяция хищников (y)",
title="Фазовый портрет системы",
color=:blue,
linewidth=1.5,
grid=true,
size=(800, 600),
legend=:topright)
```

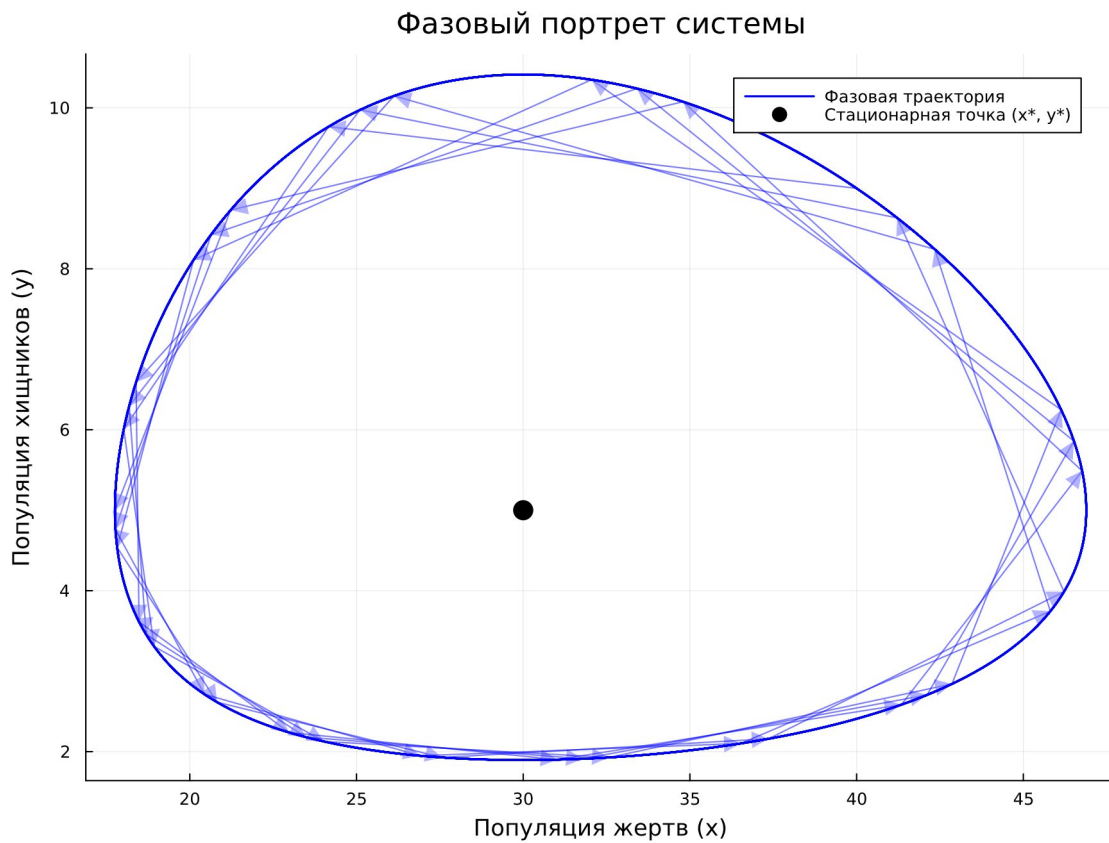


Добавление стрелок направления на фазовом портрете

```
step = 50 # шаг для отображения стрелок
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
           [df_lv.predator[i], df_lv.predator[i+step]],
           arrow=:closed, color=:blue, alpha=0.3, label=false)
end
```

Добавление стационарной точки

```
scatter!(plt2, [x_star], [y_star],
         color=:black, markersize=8, label="Стационарная точка (x*, y*)")
```



Изоклины (нулевого роста)

```
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range) # y-изоклина (dy/dt = 0)
plot!(plt2, x_range, y_nullcline,
color=:red, linestyle=:dash, linewidth=1.5, label="Изоклина хищников (dy/dt=0)")
y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range))) #x-изоклина (dx/dt = 0)
plot!(plt2, x_nullcline, y_range,
color=:green, linestyle=:dash, linewidth=1.5, label="Изоклина жертв (dx/dt=0)")
```

Фазовый портрет системы

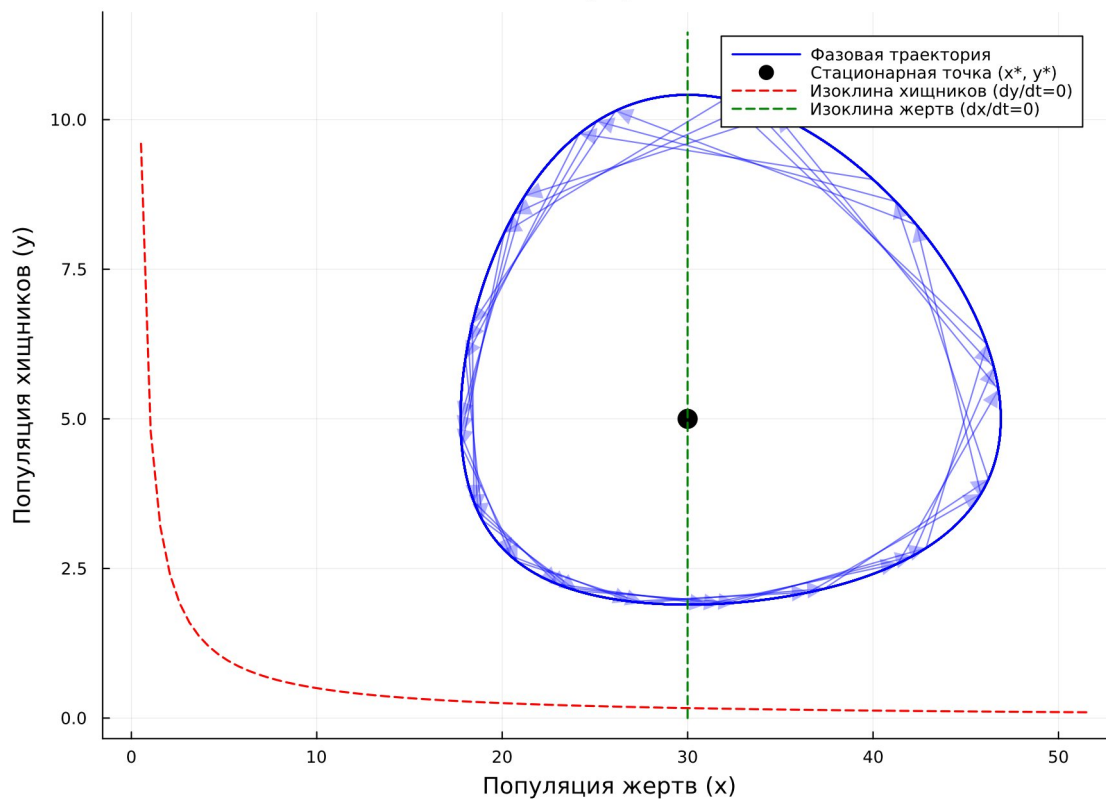


График 3: Производные (скорости изменения)

```
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
label=[L"dx/dt" L"dy/dt"],
xlabel="Время",
ylabel="Скорость изменения",
title="Производные популяций",
linewidth=1.5,
legend=:topright,
grid=true,
size=(900, 400),
color=[:green :red])
hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=false)
```

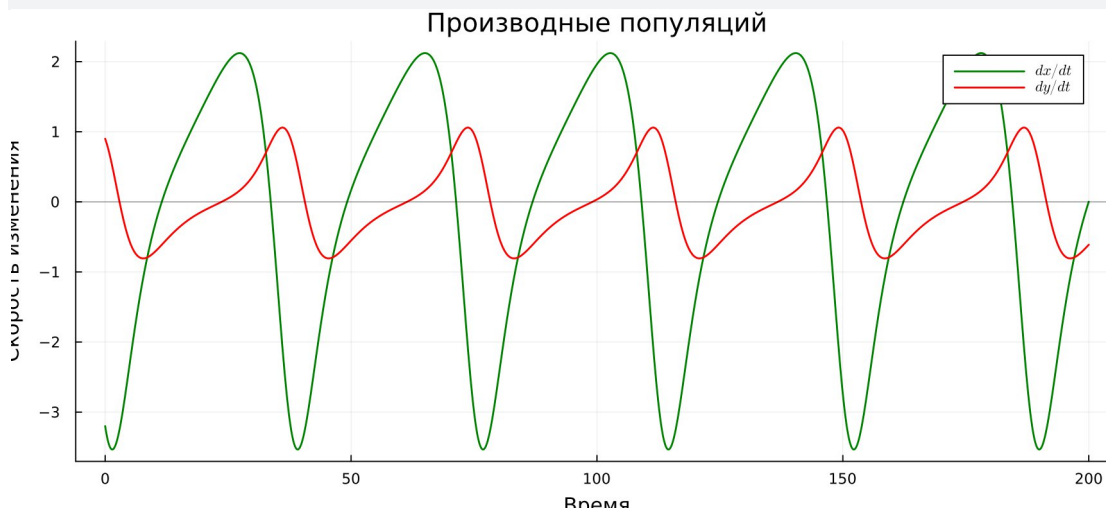


График 4: Относительные изменения (в %)

```
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator .* 100
plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
label=[L"dx/dt / x (%)" L"dy/dt / y (%)"],
xlabel="Время",
ylabel="Относительное изменение, %",
title="Относительные темпы роста",
linewidth=1.5,
legend=:topright,
grid=true,
size=(900, 400),
color=[:green :red])
```

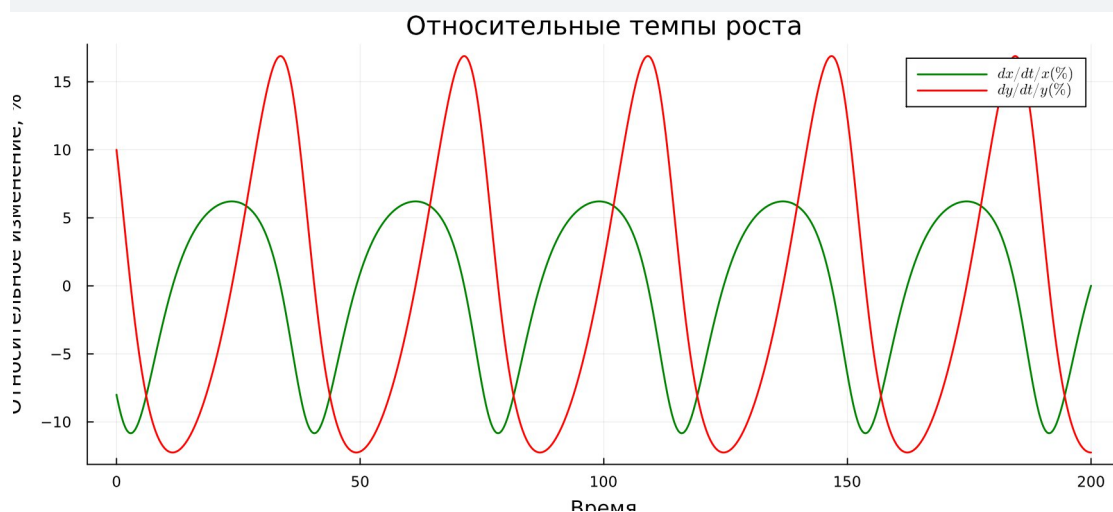


График 5: Спектральный анализ (быстрое преобразование Фурье)

```
function compute_fft(signal, dt)
n = length(signal)
spectrum = abs(rfft(signal))
freq = rfftfreq(n, 1/dt)
return freq, spectrum
end
```

compute_fft (generic function with 1 method)

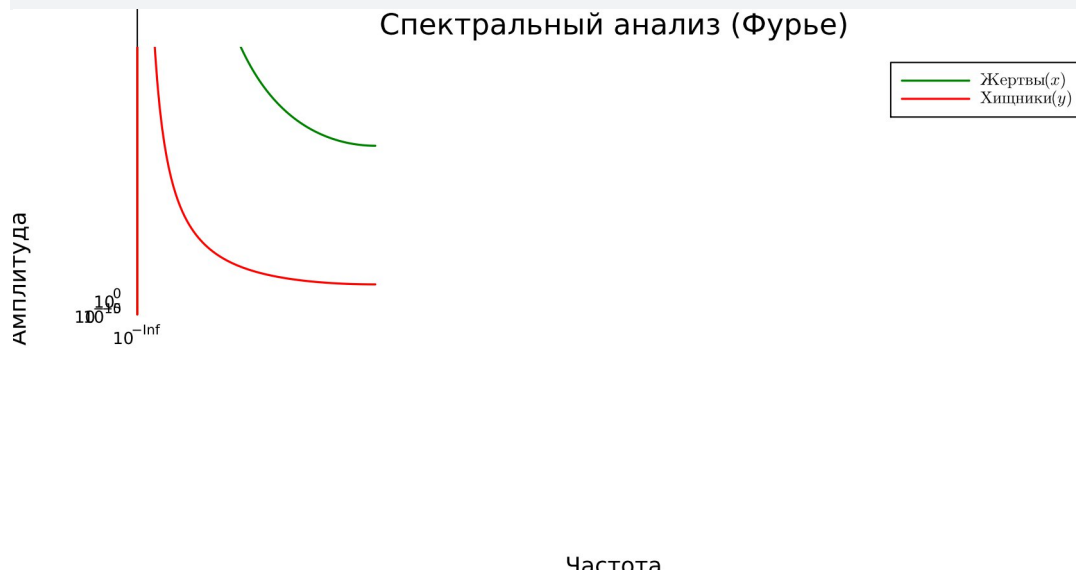
Вычисление периодов колебаний

```
freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv.prey), dt_lv)
freq_predator, spectrum_predator = compute_fft(df_lv.predator .- mean(df_lv.predator),
dt_lv)
plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
label=[L"Жертвы (x)" L"Хищники (y)"],
xlabel="Частота",
ylabel="Амплитуда",
title="Спектральный анализ (Фурье)",
linewidth=1.5,
```

```

xscale=:log10,
yscale=:log10,
legend=:topright,
grid=true,
size=(800, 400),
color=[:green :red]

```



```

└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105

```

Нахождение доминирующих частот

```
if length(spectrum_prej) > 0
idx_prej = argmax(spectrum_prej[2:end]) + 1 # пропускаем нулевую частоту
dominant_freq_prej = freq_prej[idx_prej]
period_prej = 1/dominant_freq_prej
println("\nДоминирующая частота колебаний жертв: ", round(dominant_freq_prej,
digits=4), " Гц")
println("Период колебаний жертв: ", round(period_prej, digits=2), " единиц времени")
end
```

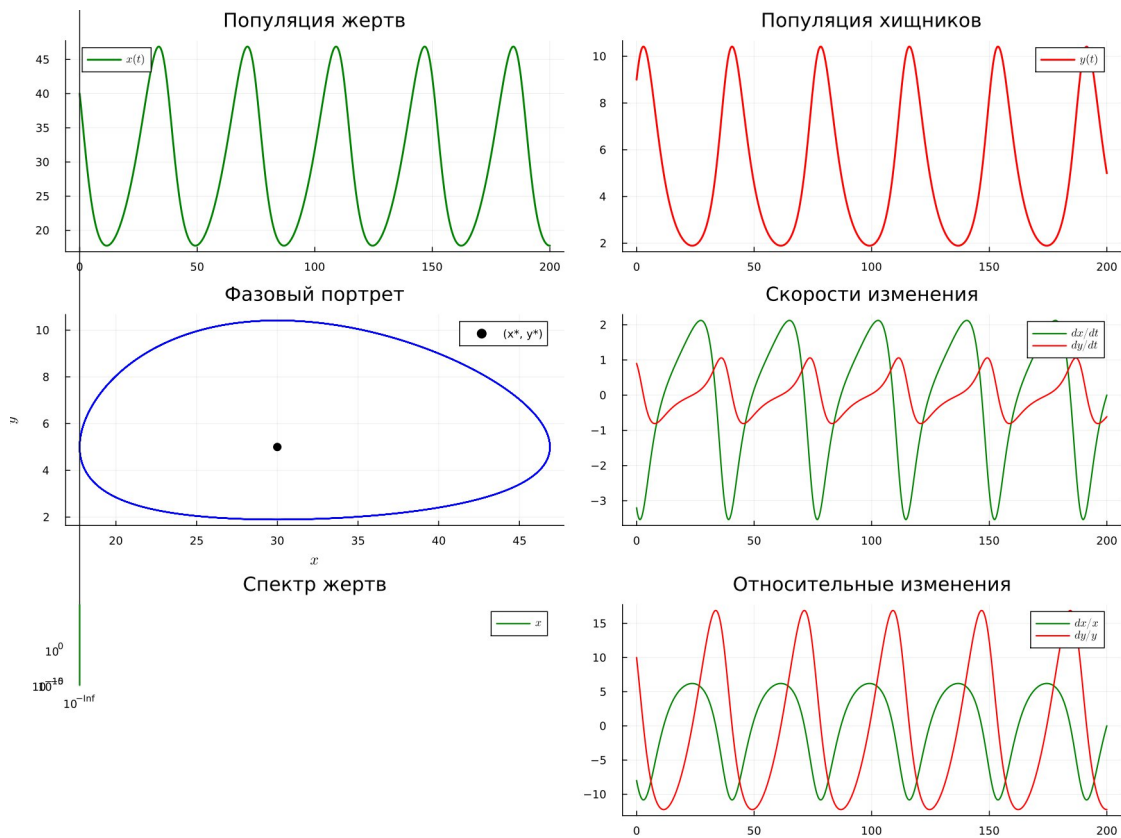
Доминирующая частота колебаний жертв: 0.2499 Гц

Период колебаний жертв: 4.0 единиц времени

График 6: Компактная панель всех графиков

```
plt6 = plot(layout=(3, 2), size=(1200, 900))
plot!(plt6[1], df_lv.t, df_lv.prej, label=L"x(t)", color=:green, linewidth=2,
title="Популяция жертв", grid=true)
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red, linewidth=2,
title="Популяция хищников", grid=true)
plot!(plt6[3], df_lv.prej, df_lv.predator, label=false, color=:blue, linewidth=1.5,
title="Фазовый портрет", xlabel=L"x", ylabel=L"y", grid=true)
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, label="(x*, y*)")
plot!(plt6[4], df_lv.t, [df_lv.dprej_dt df_lv.dpredator_dt],
label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
title="Скорости изменения", grid=true, legend=:topright)
plot!(plt6[5], freq_prej, spectrum_prej, label=L"x", color=:green, linewidth=1.5,
title="Спектр жертв", xscale=:log10, yscale=:log10, grid=true)
plot!(plt6[6], df_lv.t, [df_lv.prej_pct_change df_lv.predator_pct_change],
label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
title="Относительные изменения", grid=true, legend=:topright)

└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utills.jl:105
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utills.jl:105
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
```



Warning: No strict ticks found

@ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194

Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale

@ Plots ~/.julia/packages/Plots/AE0LB/src/utills.jl:105

Анализ результатов

```
println("\n" * "="^60)
println("Анализ результатов")
println("="^60)
println("\nОсновные статистики:")
println("Жертвы: min = ", round(minimum(df_lv.prey), digits=2),
", max = ", round(maximum(df_lv.prey), digits=2),
", mean = ", round(mean(df_lv.prey), digits=2))
println("Хищники: min = ", round(minimum(df_lv.predator), digits=2),
", max = ", round(maximum(df_lv.predator), digits=2),
", mean = ", round(mean(df_lv.predator), digits=2))
```

Анализ результатов

Основные статистики:

Жертвы: min = 17.75, max = 46.89, mean = 29.71

Хищники: min = 1.9, max = 10.41, mean = 5.2

Упрощенный анализ колебаний без поиска максимумов Вместо сложного анализа сдвига фаз, просто посчитаем основные характеристики Находим время первого пика жертв (простой алгоритм)

```
function find_first_peak(signal, time)
for i in 2:length(signal)-1
if signal[i] > signal[i-1] && signal[i] > signal[i+1]
return time[i], signal[i]
end
end
return NaN, NaN
end
peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv.t)
peak_time_predator, peak_value_predator = find_first_peak(df_lv.predator, df_lv.t)
if !isnan(peak_time_prey) && !isnan(peak_time_predator)
phase_shift = peak_time_predator - peak_time_prey
println("\nАнализ колебаний:")
println("Первый пик жертв: время = ", round(peak_time_prey, digits=2),
", значение = ", round(peak_value_prey, digits=2))
println("Первый пик хищников: время = ", round(peak_time_predator, digits=2),
", значение = ", round(peak_value_predator, digits=2))
println("Сдвиг фаз (хищники отстают): ", round(phase_shift, digits=2))
end
```

Анализ колебаний:

Первый пик жертв: время = 33.7, значение = 46.89

Первый пик хищников: время = 2.9, значение = 10.41

Сдвиг фаз (хищники отстают): -30.8

Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
```

└ Warning: No strict ticks found

└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194

└ Warning: No strict ticks found

└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194

└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale

└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105

└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale

└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105

└ Warning: No strict ticks found

└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194

└ Warning: No strict ticks found

```
L @ PlotUtils ~/julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
L @ Plots ~/julia/packages/Plots/AE0LB/src/utils.jl:105

"/home/srluipp/plots/lv_panel.png"
```

Дополнительный анализ: чувствительность к параметрам

```
println("\n\n" * "="^60)
println("Анализ чувствительности")
println("="^60)

=====
Анализ чувствительности
=====
```

Функция для анализа влияния параметров

```
function analyze_parameter_sensitivity(param_index, values, param_name)
println("\nАнализ чувствительности к параметру: ", param_name)
results = []
for val in values
p_test = copy(p_lv)
p_test[param_index] = val
prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_test)
sol_test = solve(prob_test, dt = dt_lv)
prey_end = sol_test.u[end][1]
predator_end = sol_test.u[end][2]
push!(results, (param=val, prey=prey_end, predator=predator_end))
println(" $(param_name)=$(val): жертвы=$(round(prey_end,2)),
хищники=$(round(predator_end,2))")
end
return results
end
```

analyze_parameter_sensitivity (generic function with 1 method)

Анализ чувствительности к ключевым параметрам

```
if false # Установите в true для выполнения анализа
println("\n1. Влияние скорости размножения жертв ( $\alpha$ ):")
analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], " $\alpha$ ")
println("\n2. Влияние смертности хищников ( $\gamma$ ):")
analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], " $\gamma$ ")
end
println("\nМоделирование завершено успешно!")
```

Моделирование завершено успешно!

5. Выводы

В ходе лабораторной работы я изучила модель SIR, модель Лотки–Вольтерры, узнала ее практическое применение, изучила трехпараметрическую модель и ее практическую ценность.

Список литературы